

[Abusive Comments Detection on Social Media Platforms Using NLP with BERT and Transfer Learning]

Project Report Submitted

To

Dr. D. Y. Patil Vidyapeeth, Centre for Online Learning, Pune



In Partial Fulfillment of the Requirement for the Award of

the Degree of

MASTER OF BUSINESS ADMINISTRATION

BY

[Shubham Chaudhary] [ERP ID: 241111543]

[PRN No.: 2405020015468]

Name of the Internal Mentor

[Vikas Kumar Atray]

[MBA in AI&ML]

Dr. D. Y. Patil Vidyapeeth, Centre For Online Learning, Pune.

2024-2026

Declaration

I, the undersigned, hereby declare that the project report entitled “[**Abusive Comments Detection on Social Media Platforms Using NLP with BERT and Transfer Learning**],” submitted by me to **Dr. D. Y. Patil Vidyapeeth, Centre for Online Learning, Pimpri, Pune**, in partial fulfillment of the requirements for the award of the degree of **Master of Business Administration (MBA)**, is my original work.

The conclusions and findings presented in this report are based on the information and resources collected from reliable sources and compiled by me. This work has not been submitted to any other university or institution for the award of any degree, diploma, or fellowship.

Place: Pune

[Shubham Chaudhary]

[Submitted Date]



CERTIFICATE

This is to certify that the Project Report entitled “[**Abusive Comments Detection on Social Media Platforms Using NLP with BERT and Transfer Learning**],” which is being submitted herewith for the award of **Master of Business Administration (MBA)** of **Dr. D. Y. Patil Vidyapeeth, Centre for Online Learning, Pimpri, Pune**, is the result of the original research work carried out by **Dr. /Mr. /Ms. [Shubham Chaudhary]** having **[PRN No.: 2405020015468]** under my supervision.

To the best of my knowledge, the Project Report submitted by the student is in accordance with the guidelines prescribed by the Institute and the University.

Internal Mentor

[Vikas Kumar Atray]



Director DPU COL

[Dr. Safia Farooqui]

Accepted [Date]

Date:

ACKNOWLEDGEMENTS

I take this opportunity as a privilege to express my deep sense of gratitude to **Dr. D. Y. Patil Vidyapeeth, Centre for Online Learning, Pune**, for providing me with the platform and resources to pursue my research.

I extend my sincere thanks to **Dr. Safia Farooqui, Director, Dr. D. Y. Patil Vidyapeeth, and Centre for Online Learning, Pimpri, Pune**, for her continual support in fostering active learning and for her constant encouragement to achieve excellence. I also appreciate the dedicated efforts of the academic team, headed by Dr. Rajendra Takale, along with all the mentors, whose guidance and inspiration have greatly contributed to the completion of this research work.

I am deeply indebted to my **research mentor, Vikas Kumar Atray**, Dr. D. Y. Patil Vidyapeeth, Centre for Online Learning, Pune. Without his/her invaluable guidance, constant motivation, and insightful suggestions, the completion of this project would not have been possible. I am sincerely grateful for the knowledge and research orientation imparted to me under his/her mentorship.

I also wish to extend my heartfelt thanks to all the teaching and non-teaching staff members of the Centre for Online Learning for their unwavering support and cooperation throughout this journey.

Finally, I am profoundly thankful to my family members, relatives, and friends for their encouragement, understanding, and support, which enabled me to complete this research successfully.

Place: Pune

[Shubham Chaudhary]



[Submitted Date]

TABLE OF CONTENTS

Chapter no.	Content	Page no.
	DECLARATION BY STUDENT	II
	CERTIFICATE BY THE GUIDE	III
	ACKNOWLEDGMENT	IV
	TABLE OF CONTENTS	V-VII
	LIST OF FIGURES	VIII-IX
	LIST OF TABLES	X
	LIST OF ABBREVIATIONS	XI-XII
	ABSTRACT	XIII-XIV
I	Chapter 1: Introduction	1-6
II	Chapter 2: Literature Review	7-8
III	Chapter 3: Theoretical Framework	9-17
IV	Chapter 4: Methodology and System Design	18-29
V	Chapter 5: Results and Discussion	30-44
VI	Chapter 6: Conclusion and Future Work	45-54
	BIBLIOGRAPHY	55-57
	ANNEXES	58-61

Layout of Chapters Table of Contents: Includes all subsections of each chapter, appendices (if applicable), and page numbers.

Chapter 1: Introduction

1.1 Background and Motivation

- 1.2 Problem Statement
- 1.3 Project Objectives
- 1.4 Scope and Limitations of the Study
- 1.5 Significance of the Project
 - 1.5.1 The Business Value of Automated Moderation
- 1.6 Organization of the Report

Chapter 2: Literature Review

- 2.1 Introduction to Abusive Language on Social Media
- 2.2 Evolution of NLP in Text Classification
- 2.3 Traditional Machine Learning vs. Deep Learning Approaches
- 2.4 The Shift to Transformer Architectures
- 2.5 Related Work in Toxicity and Hate Speech Detection
- 2.6 Research Gaps Identified

Chapter 3: Theoretical Framework

- 3.1 Natural Language Processing (NLP) Fundamentals
- 3.2 Word Embedding (Word2Vec, GloVe) vs. Contextual Embedding
- 3.3 Understanding BERT (Bidirectional Encoder Representations from Transformers)
 - 3.3.1 Transformer Architecture (Attention Mechanism)
 - 3.3.2 Pre-training Objectives (Masked Language Modeling, Next Sentence Prediction)
- 3.4 Evaluation Metrics for Imbalanced Datasets (Precision, Recall, F1-Score, ROC-AUC)
- 3.5 Regulatory Landscape and Data Privacy (DPDP & GDPR)

Chapter 4: Methodology and System Design

- 4.1 Proposed System Architecture
- 4.2 Dataset Selection and Description (e.g., Wikipedia Toxic Comments, Twitter Hate Speech datasets)
- 4.3 Exploratory Data Analysis (EDA)
- 4.4 Data Preprocessing
 - 4.4.1 Text Cleaning (Removing URLs, Mentions, Special Characters)
 - 4.4.2 BERT Tokenization (Word Piece Tokenizer)
 - 4.4.3 Padding, Truncation, and Attention Masks
- 4.5 Model Implementation

4.5.1 Loading Pre-trained BERT Model

4.5.2 Adding the Classification Layer

4.5.3 Fine-Tuning Process

4.6 Experimental Setup (Hyperparameters, Optimizer, Loss Function)

Chapter 5: Results and Discussion

5.1 Hardware and Software Environment

5.2 Training and Validation Performance

5.3 Model Evaluation

5.3.1 Confusion Matrix Analysis

5.3.2 Classification Report (Accuracy, Precision, Recall, F1-Score)

5.4 Comparison with Baseline Models (e.g., Logistic Regression, LSTM, CNN)

5.5 Error Analysis (Analyzing False Positives and False Negatives)

5.6 Ethical Considerations and Mitigating Algorithmic Bias

5.7 Financial Feasibility and Cloud Infrastructure Economics

Chapter 6: Conclusion and Future Work

6.1 Summary of Contributions

6.2 Conclusion

6.3 Future Enhancements

6.3.1 Multilingual Abuse Detection (using mBERT or XLM-RoBERTa)

6.3.2 Real-time Browser Extension/API Integration

6.3.3 Handling Sarcasm and Implicit Abuse

6.3.4 Bridging the Gap: MLOps and Real-Time Infrastructure

6.3.5 Combating Concept Drift and Continuous

Bibliography (APA Style): Books, Journals, Reports, News articles, Webliography, etc.

- A bibliography is a list of all the sources you used to generate your ideas about the topic, including those cited in your assignment as well as those you did not cite.
- Proper websites (not search engines) need to be mentioned for the data used from them.

Annexures: Questionnaire, List of Interview Questions, Annual Reports, or any other company documents.

LIST OF FIGURES

Serial No.	Topic Name	Page No.
1	Figure 1: Systematic literature review process	13
2	Figure 2: Flowchart for the methodology	18
3	Figure 3: Distribution of Label Occurrences	20
4	Figure 4: Architecture of BERT model	22
5	Figure 5: Binary and Multiclass Stack Architecture	25
6	Figure 6: Model Accuracy curves on HASOC Hindi Dataset for detection of abusive comments	33
7	Figure 7: Model Loss curves on HASOC Hindi Dataset for detection of abusive comments	34
8	Figure 8: Confusion Matrix on HASOC Hindi Dataset for abusive comment detection	35
9	Figure 9: Model Accuracy curves on HASOC Hindi Dataset for detection of sentiments of abusive comments	38
10	Figure 10: Model Loss curves on HASOC Hindi Dataset for detection of the sentiment of abusive comments	39
11	Figure 11: Confusion Matrix on HASOC Hindi Dataset for detection of sentiments in abusive comments	40
12	Figure 12: Confusion Matrix of ensemble models on HASOC Hindi Dataset	41

Serial No.	Topic Name	Page No.
13	Figure 13: Comparative analysis of different models with the ensemble model	41
14	Figure B.1: Data Cleaning and Tokenization Pipeline	59
15	Figure B.2: Model Setup and Device Configuration	60

LIST OF TABLES

Serial No.	Topic Name	Page No.
1	Table 1: Systematic literature review protocol	13-14
2	Table 2: Dataset Overview: Abusive Comment Classification and Sentiment Breakdown	19
3	Table 3: Analysis of abusive comment detection by using various BERT models on Hasoc Hindi Dataset	32
4	Table 4: Analysis of sentiments by using Various BERT Models	36
5	Table 5: Results of Ensemble model	36
6	Table 5: A Comparison of the Proposed Methodology with the State-of-the-art Methods	42
7	Table 6: The variants of BERT-based models for NLP tasks	46-47
8	Table 7: The applied hyperparameters excerpted from the reviewed papers	49-50
9	Table 8: The applied evaluation metrics excerpted from the reviewed papers	51-52
10	Table C.1: Sample Outputs from the Test Set	61

LIST OF ABBREVIATIONS

Serial No.	Abbreviation	Definition
1	AI	Artificial Intelligence
2	BERT	Bidirectional Encoder Representations from Transformers
3	Bi-LSTM	Bidirectional Long Short-Term Memory
4	BNB	Bernoulli Naive Bayes
5	ET	Extra Tree Classifier
6	FN	False Negative
7	FP	False Positive
8	GNN	Graph Neural Networks
9	GPT	Generative Pre-trained Transformer
10	HASOC	Hate Speech and Offensive Content Identification
11	HITL	Human-in-the-Loop
12	LLM	Large Language Models
13	MLM	Masked Language Modelling
14	NLG	Natural Language Generation
15	NLP	Natural Language Processing

Serial No.	Abbreviation	Definition
16	NLU	Natural Language Understanding
17	NSP	Next Sentence Prediction
18	OSN	Online Social Networks
19	RNN	Recurrent Neural Network
20	SLR	Systematic Literature Review
21	SVM	Support Vector Machine
22	TF-IDF	Term Frequency-Inverse Document Frequency
23	TN	True Negative
24	TP	True Positive

Abstract

Abusive Comments Detection on Social Media Platforms Using NLP with BERT and Transfer Learning

By

[Shubham Chaudhary]

MBA SEM IV [AI & ML]

Project Mentor: [Vikas Kumar Atray]

Specializations: [Artificial Intelligence & Machine Learning]

In today's digital age, social media plays a key role in how people communicate and share their thoughts and opinions. However, this online environment also faces challenges, particularly with rude and offensive language. This research tackles both issues by focusing on sentiment analysis and the detection of abusive comments on social media. The research area in the Natural Language Processing (NLP) domain has made major advances in recent years. The Bidirectional Encoder Representations from Transformers (BERT) and its derivative models have been at the vanguard, gaining notice for their exceptional performance across a variety of NLP applications. As a response to this context, this study aims to conduct a systematic literature review on current research in BERT-based models to describe their characteristic variations on three frequently demanded natural language processing (NLP) tasks, i.e., text classification, question answering, and text summarization. This study employed a systematic literature review method as prescribed by Kitchenham. We collected 4,120 papers from publications indexed. Our search was conducted using Scopus and Google Scholar, and we found 41 results that fit our criteria. We looked at the criteria and picked some for a closer look. Our review led to three main findings. First, to select appropriate models for particular NLP tasks, three primary concerns should be considered: i) the type of NLP problem to be resolved (i.e., the NLP task to be served); ii) the specific domain to be handled (such as financial, medical, law/legal, or others); and iii) the intended language to be applied (such as English or others). Second, learning rate, batch size, and the type of optimizer were the three most considered hyperparameters to be properly arranged in model training. Third, the most widely used metrics for text classification tasks were F1-score, accuracy, precision, and sensitivity (recall), while question answering, and text summarization tasks mostly used the Exact Match and ROUGE, respectively. The study uses data from the Hate Speech and Offensive

Content Identification (HASOC) dataset, which includes content from 2019 to 2021, specifically to identify hate speech in English across various social media platforms. To classify comments as either abusive or non-abusive, the research uses several BERT models, such as mBERT, DistilBERT, RoBERTa, HateBERT, and IndicBERT. It also includes a detailed sentiment analysis of the offensive comments. The research introduces a stacked ensemble approach for both binary (abusive vs. non-abusive) and multiclass (hate, offensive, and profane) classification, combining the predictions from mBERT, HateBERT, and IndicBERT. Additionally, the study offers an integrated method for detecting abusive comments and analyzing sentiment using a multi-output model. The proposed ensemble model achieves 94% accuracy in binary classification, with precision, recall, and F1 scores all close to 94%. For multiclass classification, the model reaches 93% accuracy, along with precision, recall, and F1 scores of 91%, 92%, and 92%, respectively. The research also includes a comparison with other advanced techniques to show how effective the suggested approach is.

Keywords: BERT, NLP, NLP task, systematic literature review, transformer.

Chapter 1

Introduction

1.1 Background and Motivation

Social media has become a big part of our lives, making it easy for us to connect with lots of people and share our thoughts, ideas, and experiences. This helps us feel validated and understood. Social media apps are powerful and influential, and they're used in many areas of our lives. But as technology keeps changing, it also brings new challenges. While social media is great for community engagement and sharing knowledge, it also raises concerns about privacy, security, and the spread of false information and online abuse. Despite its benefits, social media has a darker side, with mean comments, cyberbullying, and negative behaviors. The people in charge of online communities have a tough job making sure users follow the rules. But with so many messages and interactions, it's getting harder to keep up. Sometimes, people from different backgrounds use hurtful language to attack each other. Many users feel more comfortable expressing themselves in their own language, rather than English, which highlights the need for systems that can detect abusive language in many different languages. It's not just about finding abusive comments but also understanding why people say mean things. Looking at the psychological and emotional reasons behind these actions can help us figure out what triggers people to make unpleasant remarks. We also need to analyze comments to see if they contain hate speech or profanity. This is a crucial task, especially as social media continues to grow and evolve. By understanding these issues, we can work towards creating a safer and more positive online community. The impact of social media on our lives is complex, and it's essential to consider both the benefits and the drawbacks. On the one hand, social media has made it easier for us to connect with others and share our experiences. On the other hand, it has also created new challenges, such as the spread of false information and online abuse. To address these challenges, we need to develop systems that can detect and prevent abusive language, as well as promote positive online behaviors. Ultimately, the key to creating a positive online community is to understand the complexities of social media and its impact on our lives. By acknowledging both the benefits and the drawbacks, we can work towards creating a safer and more supportive online environment. This requires a multifaceted approach that involves not only technology but also education and awareness. By working together, we can promote positive online behaviors and create a more compassionate and inclusive online community. In conclusion, social media is a powerful tool that has both positive and negative effects on our lives. While it has made it easier for us to connect with others and share our

experiences, it has also created new challenges, such as the spread of false information and online abuse. To address these challenges, we need to develop systems that can detect and prevent abusive language, as well as promote positive online behaviors. By understanding the complexities of social media and its impact on our lives, we can work towards creating a safer and more supportive online environment. The field of natural language processing (NLP) has become increasingly significant within the realms of artificial intelligence (AI) and machine learning. Numerous NLP tasks, such as text classification (including sentiment analysis), machine translation, question answering, text summarization, information extraction, instruction following, machine reading comprehension, and image captioning, can now be effectively managed by deep learning algorithms. Among these tasks, there is a growing need for text classification, question answering, and text summarization, as their application in specific contexts often requires tailored implementations. This contrasts with machine translation, which is generally more universal, allowing users to utilize existing tools without extensive customization for their specific needs.

1.2 Problem Statement

Social media can have a big impact on our mental health. Lots of people get harassed by strangers or anonymous users, which can be really scary and unsettling. When people leave mean comments on social media, it can be really hurtful, especially for people who are already vulnerable. These comments can spread hate and make the person feel anxious, depressed, and left out. Even if someone is just joking around, using mean language can still cause problems and lead to conflicts. Mean comments on social media can really damage our mental health. They can make us feel stressed, lower our self-esteem, and make us feel lonely. We need to understand how these feelings work so we can see the bigger picture of how online negativity affects our relationships and mental well-being. Overall, online abuse can affect our daily lives and hurt our mental health in the long run, not just right away. For example, a study found that 41% of people who were bullied online developed social anxiety, 37% became depressed, and 26% had thoughts of suicide. These numbers show just how important it is to do something about mean comments online. We need to find ways to stop online abuse and make the internet a safer, more supportive place for everyone. It's not just about the people being bullied, either. When we see mean comments online, it can affect us too. We might start to feel anxious or stressed, or we might even start to doubt ourselves. That's why it's so important to be kind and supportive online and to stand up for people who are being bullied. By working together, we can create a more positive and uplifting online community. So, what can we do to make a difference? We can start by being more mindful of how our words affect others online. We can also support people who are being bullied and report mean comments

when we see them. By taking these small steps, we can help create a safer, more supportive online environment for everyone. And that's good for our mental health, too. A key advancement in NLP research in recent years has been the development of the Bidirectional Encoder Representations from Transformers (BERT) model by researchers at Google AI Language. BERT utilizes a transformer architecture as a pre-trained language model that derives contextual word representations from large amounts of unlabeled text data. Through innovative training methods like Masked Language Modeling (MLM) and Next Sentence Prediction (NSP), BERT has achieved remarkable results across a variety of NLP tasks, setting new standards for excellence. The success of BERT has generated significant interest in the research community, leading to the creation of numerous variations and model-specific modifications of BERT.

1.3 Project Objectives

Together with the Generative Pre-trained Transformer (GPT), BERT has emerged as a primary foundational model among various Transformer-based models that can be fine-tuned for different NLP applications. Its bidirectional approach allows it to be context-aware at the level of individual words, unlike GPT, which operates with token-level awareness. This capability makes BERT particularly suitable for natural language understanding (NLU) tasks, although GPT may be more effective for natural language generation (NLG) tasks. Another factor contributing to BERT's popularity is its classification as an open source, which makes it freely available for use and further development. Given these considerations, we chose to concentrate our literature review on BERT-based models rather than a broader category of Transformer-based models. Despite extensive research and application of BERT in various NLP tasks, there remains a necessity to compile and critically evaluate the existing literature to achieve a thorough understanding of its effectiveness, limitations, and possible avenues for enhancement.

The lack of labeled data in Hindi is a big problem. This is because most social media platforms are in English, making it hard to get enough data to train good models in Hindi. This doesn't just affect how much data we have, but also how diverse and good it is. For languages that don't have a lot of resources, it's tough to find enough labeled data to train machine learning models. The datasets we do have might be too small or not diverse enough, which can lead to biases in the models. Most of the best models and techniques are made for English, which ignores the unique features of Hindi and other local languages. To make these models work well with Hindi, we need to put in a lot of work to collect data, preprocess it, engineer features, and train the models. But even then, the performance might not be great. Understanding abusive language and sentiment can be tricky,

especially for models trained on data from other languages or cultures. It's crucial to make sure the detection process is sensitive to cultural differences. If models are trained on datasets that are imbalanced or biased, they might make predictions that are unfair or discriminatory, especially for groups that are already underrepresented. To deal with these bias and fairness issues, we need to be careful about how we collect and prepare data, design models, and evaluate their performance. We have to make sure our models are fair and don't discriminate against anyone. For example, when we're training models to detect abusive language, we need to make sure they can understand the nuances of Hindi and not mistakenly flag something as abusive just because it's from a certain culture or region. This requires a deep understanding of the language and its cultural context. By being more careful and thoughtful in how we train our models, we can make sure they're fair, accurate, and helpful for everyone. Overall, the scarcity of labeled data in Hindi is a challenge we need to address. We need to work harder to collect and prepare good data and to develop models that are sensitive to the unique features of Hindi and other local languages. By doing so, we can create models that are fair, accurate, and helpful for everyone, regardless of their language or cultural background. The task of detecting abusive comments on various platforms is crucial. The study shows an ensemble strategy for both abusive comment detection and sentiment analysis by using the stack generalization method. The following are the research contributions:

- a. The study utilizes the Hate Speech and Offensive Content Identification (HASOC) datasets, covering the period from 2019 to 2021.
- b. The advanced pre-trained Bidirectional Encoder Representations from Transformers (BERT) language model has been used to detect abusive comments and analyze the underlying sentiments.
- c. The research compares and analyzes different BERT variants, including mBERT, IndicBERT, DistilBERT, RoBERTa, and HateBERT.
- d. The mBERT, IndicBERT, and HateBERT models have been used in a stacked ensemble framework for the identification of abusive comments and the underlying sentiment analysis. Subsequently, a multi-output model has been employed to integrate the ensembles and obtain results for both tasks simultaneously.
- e. Common evaluation metrics like recall, accuracy, precision, and F1 score have been used to evaluate the effectiveness of the suggested models.

1.4 Scope and Limitations of the Study

BERT's capacity to capture contextual information and its strong performance across numerous NLP tasks have made it a transformative force in the natural language processing field. However, it does have its drawbacks. A significant challenge linked to BERT is its computational demands and resource usage, which can hinder its implementation in environments with limited resources or on edge devices. Furthermore, like many language models, BERT may exhibit biases and inconsistencies, posing challenges in practical applications. A systematic literature review can offer fresh insights into the current research landscape, highlight gaps in the research, and guide future investigations.

1.5 Significance of the Project

This systematic literature review aims to thoroughly examine existing research on BERT-based models to highlight their distinct variations across three commonly sought natural language processing tasks: text classification, question answering, and text summarization. These tasks encompass both "discriminative" AI in the case of text classification and "generative" AI for question answering and text summarization. By analyzing and synthesizing results from pertinent studies, this review seeks to provide a clearer understanding of the effective applications of BERT-based models in these three highly requested NLP tasks.

1.5.1 The Business Value of Automated Moderation

While the technical achievements of Natural Language Processing are profound, the primary motivation for this project is deeply rooted in business sustainability. For social media platforms, online communities, and e-commerce forums, user retention is the ultimate metric of success. Rampant toxicity, cyberbullying, and unmoderated hate speech create a hostile user environment, directly leading to user churn. When marginalized users or casual participants are repeatedly subjected to abuse, they simply leave the platform, taking their engagement and associated ad-revenue with them.

Traditionally, platforms have relied on vast teams of human moderators to manually review flagged content. This approach is not only financially unsustainable at scale, but it also exposes human workers to severe psychological trauma, leading to incredibly high employee turnover and operational bottlenecks.

By implementing the stacked ensemble model developed in this research, platforms can achieve a highly automated, scalable "first line of defense." With a binary classification accuracy of 94%, the

model can autonomously filter out the vast majority of explicitly abusive content before it reaches the end-user. This drastic reduction in manual review volume allows businesses to reallocate their human moderation budget toward highly complex, ambiguous edge cases—thereby lowering operational expenditure (OpEx) while simultaneously protecting the mental well-being of both their users and their workforce.

1.6 Organization of the Report

However, despite being such a revolutionary technology, there is a downside to the use of BERT, as it possesses some limitations. For example, the computational power and the resource demands are rather high when applying this technology, thus making the deployment challenging in low-resource environments. Furthermore, just like other language models, BERT suffers from biases and inconsistencies, which pose a problem in practical applications. A systematic literature review provides researchers with an insight into the state-of-the-art in the relevant field of study, thus indicating possible research gaps and future research directions.

Chapter 2

Literature Review

2.1 Introduction to Abusive Language on Social Media

Several challenges exist when detecting abusive language on online platforms like social media and marketing. For instance, the interpretation of abusive language is dependent on context and culture. Abusive language may result in severe impacts on the individual or community at large. As such, there is a need to detect and prevent abusive language on these platforms to foster social harmony.

2.2 Evolution of NLP in Text Classification

In this research, computational approaches have been applied in offensive language detection in social media posts using MOLD_DL techniques and natural language processing. For instance, FCNN has been used for selecting the most suitable features, while classification has been performed using a combination of Bi-LSTM along with Naïve Bayes and SVM. This study has made significant contributions in moderating social media content using an efficient method for offensive language detection. It is common practice to use linear SVM and Naïve Bayes algorithms in NLP and machine learning to tackle issues related to abusive language detection on social media. According to research studies, Naïve Bayes managed 92% accuracy and 95% recall, whereas linear SVM was able to achieve 90% accuracy and 92% recall in the identification of offensive content. Lexical augmentation of BERT through a hate lexicon helped improve its efficiency in the detection of abusive language in social media posts. The different ways considered augmentation included sentence-level encoding and word-level embeddings. The performance evaluation results of BERT models enriched with lexicons have been shown to exceed the baseline model of BERT. Transfer learning has been used along with BERT to detect hate speech in social media, where the methodology of transfer learning along with BERT achieved 90% accuracy, and employing adversarial training enhanced the accuracy to 92%. The approach of reinforcement learning has been used to detect offensive comments. The technique employed the use of LSTM and reinforcement learning.

2.3 Traditional Machine Learning vs. Deep Learning Approaches

A model with layers in Urdu, that is, a low-resource language, has been designed to detect threatening language on social media sites. The layers consist of base learners such as BNB (Bernoulli Naive Bayes) and ET (Extra Tree Classifier). Supervised learning and unsupervised learning techniques were compared in a study for the first time where the focus was on recognizing abusive language in Tamil. Traditional supervised learning techniques, including Random Forest, SVM (Support Vector Machine), logistic regression, and AdaBoost were compared with human labeling.

2.4 The Shift to Transformer Architectures

On the other hand, the unsupervised learning method involving K-means clustering proved to be better than the supervised method. The objective of this research is to detect offensive language in Urdu and Roman Urdu languages by using different ML models (IBK, Logistic, NB, SVM, and JRip) and DL models (BLSTM, CNN, LSTM, and CLSTM). According to the study, the CNN outperformed other models, obtaining accuracies of 91.4% and 96.2% in Urdu and Roman Urdu languages, respectively. It is stated that both the research attention mechanism and Bi-LSTM and GRU outperform baselines in accuracy, with 84%. This research paper suggests a system based on machine learning and natural language processing techniques to help mitigate abuse cases on social media platforms. Such systems would consider Unicode emoticons and Bengali characters as well as use multinomial Naive Bayes, support vector machines with linear kernels, and convolutional neural networks with LSTM techniques.

2.5 Related Work in Toxicity and Hate Speech Detection

It was found that SVM with a linear kernel gives the highest accuracy – 78% – in detecting abuse comments in Bengali. To classify the abusive content written in Bengali, the researchers consider several machine learning algorithms and deep learning models. They include Multinomial Naive Bayes, Linear Support Vector Classifier, Artificial Neural Network, Recurrent Neural Network, Random Forest, and Logistic Regression with LSTM cells. Additionally, they develop specific rules for stemming in Bengali. A deep learning approach with an accuracy of 82.20% is obtained with RNN+LSTM. Moreover, a new data set containing 30,000 comments from social media that pose challenges for hate speech detection among the Bengali-speaking population has also been introduced. It consists of multiple annotations by different annotators and is based on seven classes. The research experiment involves many DL algorithms, including some based on pre-trained Bengali word embedding. SVM proves to be the best algorithm with an accuracy of 87.5%.

2.6 Research Gaps Identified

There's a big problem with artificial intelligence models like BERT; they're good at spotting obvious mean comments, but they struggle to detect subtle insults, sarcasm, and backhanded compliments. This is because the words used might seem harmless, but the person's true intention is actually hurtful. It's like when someone says something that sounds nice on the surface but is meant to be mean or insulting. These models have a hard time figuring out when someone is being sarcastic or passive-aggressive, and that's a major gap in their ability to understand language. Current systems heavily rely on specific derogatory keywords, making them vulnerable to "clean-text" abuse, where users employ polite or neutral vocabulary to harass or demean others.

Most of the best language models we have today are trained and tested using English language data. This shows that we really need to create systems that can work well with languages that don't have a lot of data available. Furthermore, there is a distinct lack of comprehensive research addressing "code-mixing" (the fluid alternation between two or more languages in a single comment), which is a standard communication style for bilingual users on global social media platforms.

The Generative AI Dilemma:

BERT vs. Large Language Models (LLMs) In the rapidly evolving landscape of Artificial Intelligence, a frequent question arises: Why train a specialized BERT ensemble when Large Language Models (LLMs) like OpenAI's GPT-4 or Meta's Gemini exist? These massive models possess unprecedented contextual understanding and can perform zero-shot classification out of the box. However, from an engineering and business perspective, relying on Generative AI for real-time content moderation presents several critical bottlenecks.

First, there is the fundamental difference between "Generative" and "Discriminative" AI. GPT-4 is an autoregressive model designed to generate text token by token. Asking it to simply output a binary "0" or "1" for abusive content is a massive over-utilization of its architecture. BERT, on the other hand, is a bidirectional encoder explicitly designed for discriminative tasks like classification. It reads the entire sentence at once, making it fundamentally faster at understanding context for the sole purpose of categorization.

Secondly, the latency and cost of LLMs are prohibitive for social media scale. A platform processing 10,000 comments a second cannot afford the latency of an API call to a massive

billion-parameter model, nor can it sustain the per-token financial cost. Our stacked ensemble (mBERT, HateBERT, IndicBERT) is comparatively lightweight. It can be hosted entirely on-premises or on private cloud instances, guaranteeing microsecond response times without sharing sensitive user data with third-party LLM providers.

Lastly, Generative AI models are prone to "hallucinations" and are heavily safety-aligned by their creators. An LLM might refuse to process an abusive comment entirely due to its internal safety guardrails, outputting a canned refusal message instead of accurately classifying the data. By fine-tuning our own BERT ensemble, we retain absolute control over the classification thresholds, ensuring predictable, deterministic, and cost-effective moderation.

Chapter 3

Theoretical Framework

3.1 Natural Language Processing (NLP) Fundamentals

Natural Language Processing (NLP) is a specialized branch of artificial intelligence that bridges the gap between human communication and computer understanding. The primary objective of NLP is to design algorithms and computational models capable of processing, analyzing, and deriving meaningful insights from large volumes of unstructured natural language data. A fundamental challenge in this field lies in translating the highly ambiguous, nuanced, and unstructured nature of human text into structured, mathematical formats that machines can compute. NLP is based on two important parts of language analysis. First, there's a syntax, which is about how words are arranged to make sentences and follow grammar rules. Then, there's semantics, which is all about understanding the actual meaning of the text. These two parts work together to help us make sense of language. While understanding syntax is crucial for basic parsing, grasping semantics is what allows an NLP system to differentiate between a genuinely threatening comment and a harmless, sarcastic joke.

3.2 Word Embedding (Word2Vec, GloVe) vs. Contextual Embedding

Prior to the advent of contextual models, algorithms like Word2Vec and GloVe became the industry standard for mapping words into continuous, low-dimensional vector spaces based on their co-occurrence in large text corpora. These static embedding techniques were highly successful at capturing syntactic and semantic similarities, mathematically encoding relationships so that words with similar meanings were positioned close to one another in the vector space. By transitioning from sparse Bag-of-Words representations to dense vectors, Word2Vec and GloVe significantly reduced dimensionality and allowed machine learning models to capture complex linguistic patterns. Despite their revolutionary impact, traditional word embedding suffered a critical flaw: they assigned a single, fixed vector to every word in the vocabulary, regardless of the surrounding context. Because of this static mapping, models could not handle polysemy—meaning a word like "bark" possessed the exact same mathematical representation whether it referred to a tree's outer layer or the sound a dog makes. When it comes to detecting abusive language, not being aware of the context is a big problem. This is because how bad a word often depends on the words around it, not just what the word means on its own. The paradigm shifted dramatically with

the introduction of contextual embedding, popularized by architectures like ELMo and subsequently perfected by Transformer-based models like BERT. Unlike static models that look up a fixed dictionary of vectors, contextual models generate dynamic representations by processing the entire sentence, assigning a vector to a word based on the specific tokens that precede and follow it. So, when we're talking about words in a certain context, they can have different meanings based on how they're used. This means the model can tell the difference between something like "toxic chemicals," which are harmful, and "a toxic user," which is someone who's being abusive. The way the model sees these words is unique to each situation, allowing it to understand the nuances of language. The transition from static embedding (Word2Vec, GloVe) to dynamic, context-aware embedding (BERT) is precisely what enables modern NLP systems to detect implicit abuse, sarcasm, and nuanced hate speech.

3.3 Understanding BERT (Bidirectional Encoder Representations from Transformers)

3.3.1 Transformer Architecture (Attention Mechanism)

In recent times, systematic literature reviews (SLRs) have gained recognition and are widely embraced as a credible research approach by the global scientific community, often featured in numerous esteemed journal publications. As a form of meta-research, SLRs are seen as a reputable research category and have established themselves as reliable sources of information. In this context, the present study was carried out as an SLR, following the framework proposed by Kitchenham. The aim of systematic literature reviews, also referred to as secondary studies, is to identify, gather, evaluate, analyze, synthesize, extract, and summarize pertinent research findings on specific, focused subjects, which classifies them as tertiary reviews. The process of conducting a systematic review according to the Kitchenham method involves three key stages: planning, conducting, and reporting. The specific stages for this study are illustrated in Figure 1.

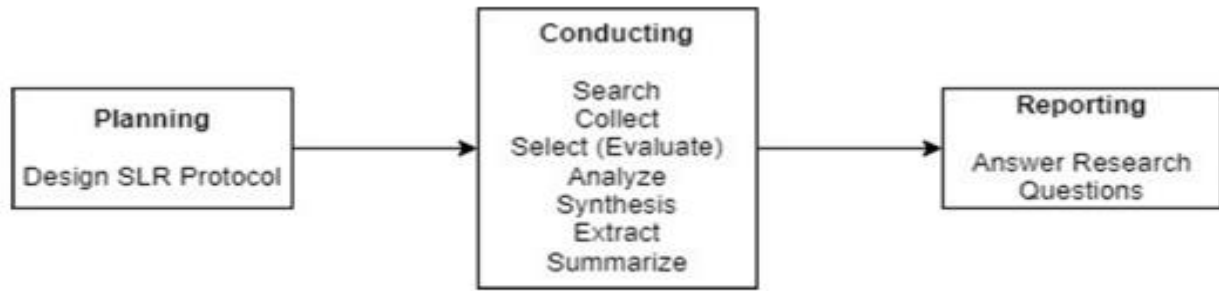


Figure 1: Systematic literature review process.

According to the Kitchenham framework, a systematic literature review (SLR) study comprises three phases, as illustrated in Figure 1. The initial phase involves planning the systematic review, during which a review protocol is developed. A crucial component of this protocol is the formulation of research questions, which will serve as the foundation for the study. The second phase entails executing the systematic review process, which includes searching for, gathering, evaluating, analyzing, synthesizing, extracting, and summarizing relevant papers based on established criteria to address the research questions. The final phase focuses on reporting the systematic review, where all aspects of the study are documented, including the answers and rationales for the research questions posed.

3.3.2 Pre-training Objectives (Masked Language Modeling, Next Sentence Prediction)

At this stage, a review protocol to conduct the study is designed with the purpose of having a systematic and structured literature review. The protocol consists of research questions, searching tools, searching terms, inclusion, exclusion, and quality assessment criteria. The protocol also establishes a data analysis and synthesis strategy as guidance in extracting and summarizing the collected papers. The protocol is described in Table 1.

Table 1: Systematic literature review protocol

SLR Protocol	Description
Research Question	1. RQ1. What type of concerns were normally considered when selecting BERT-based model variants for NLP tasks? 2. RQ2. What type of hyper parameters were mostly considered to be properly arranged for

	training the BERT-based models?
	3. RQ3. What type of evaluation metrics were normally employed for NLP tasks?
Search Tool	Publish or Perish Application.
Search Terms	“Text Classification”; “Question Answer”; “Text Summarization”.
Inclusion Criteria	1. The papers are from scientific journals published between 2019 and 2024. 2. The publications are indexed by Scopus and/or Google Scholar. 3. The publications are not in the form of journals and proceedings. 4. The paper’s theme is regarding NLP tasks that employ BERT-based models. 5. The paper is written in English.
Exclusion Criteria	1. Unpublished scientific papers between 2019 and 2024. 2. The publications are not indexed by Scopus and/or Google Scholar. 3. The publications are not in the form of journals and proceedings. 4. The paper’s theme is not regarding NLP tasks that employ BERT-based models. 5. The paper is not written in English.
Quality Assessment Criteria	1. Clarity of research objectives 2. Contains literature review, background, and research results. 3. Provides relevant conclusions. 4. Describes the method of developing or optimizing and evaluating the BERT-based model.
Data Analysis Strategy	Based on the chosen papers, each variable that is deemed suitable and relevant to answer the research questions will be identified, gathered, analyzed, and scrutinized based on the quality of the study.
Data Synthesis Strategy	A data-driven methodology, founded on the data analysis of outcomes of the papers, is used in data synthesis. To address the research questions, a table

containing a comparison list of each variable was composed for every chosen paper using a data-driven approach.

3.4 Evaluation Metrics for Imbalanced Datasets (Precision, Recall, F1-Score, ROC-AUC)

The significance of sentiment analysis in dealing with information overload, especially in the context of user-generated data on social media platforms, has been brought to light. It provides insight into the difficulties that exist in this field for scholars and examines different aspects of sentiment analysis. It also identifies issues in data sets, languages used in the text, analysis methodologies, and evaluation criteria, providing insightful comparisons between studies. Social networks online (OSNs) have been utilized to detect mental health. The key approaches employed by scholars to detect mental disorders include text mining and machine learning through OSN data. To detect aggressive behavior in social media content, machine learning is conducted, leveraging word embedding, and sentiment analysis. Evaluation shows that Random Forests are better for similar texts, while SVMs are superior for dissimilar texts. Surprisingly, the simple method can work as well as any other more complex algorithms that prove their competency in detecting aggressive behavior in social media messages. In the article, 104 mental applications were examined based on sentiment analysis of 88,125 ratings of users. This technique reveals the best classifier for predicting sentiments and highlights 21 negative and 29 positive topics in users' reviews. The author discusses the topic of X (previously called Twitter) sentiment analysis and uses NLP technology to find out what people think about tweets and the sentiments conveyed there. The task of the project has been to analyze the sentiment from tweets using natural language processing to identify emotions connected with certain issues. Three methods have been used: classification based on subjective opinions, semantic associations, and polarity classification. Emotions of lexicons and grammatical relations to a tweet of subject matter were studied. Most importantly, this methodology proved to be better than existing techniques of text sentiment analysis because of its compatibility with the unique nature of tweets. For comparison of CNN-BiLSTM findings, transfer learning together with traditional BERT has been applied to improve sentiment analysis in Bengali texts. Sentiment analysis of online customer reviews is performed using SVM in this paper. Sentiment analysis is performed for both the whole review and individual sentences to provide real-time feedback to improve user experiences. This model represents a novel approach as it combines the idea of sentiment information with traditional term frequency-

inverse document frequency (TF-IDF)-based word embedding. BiLSTM was used on the weighted vectors to enhance performance. The model conducts the sentiment analysis over several different themes, employing supervised learning with the comparison of SVM and Naive Bayes techniques. It turned out that SVM performed better after preprocessing and sentiment scoring processes were implemented. The current research utilized the Sentiment140 dataset to perform sentiment analysis and emotion detection in tweets using LSTM models, achieving high-quality results. The novelty of the work is also presented by using emoticons as a particular method of validation for supervised deep learning models in the case of data collected from X. The need for analyzing sentiments of tweets on platform X has been stated, owing to the increase in the generation of user content. A new feature ensemble method has been used, which has included several components, including semantic, lexical, word type, sentiment polarity, and position, especially for tweets containing fuzzy sentiment. The deep learning-modified neural network algorithm has been utilized for the effective process of sentiment analysis on X datasets, obtaining an efficiency level of 91.65%. Abusive comment detection techniques have primarily been concentrated around high-resource languages, such as English; as a result, low-resource languages, like Hindi, suffer from an absence of labeled data and specific pre-trained models. For overcoming these limitations, the proposed methodology has considered several variations of BERT, namely IndicBERT and mBERT, trained specifically on Indian languages. Also, most studies have considered the problem of abusive comments detection and sentiment analysis separately, ignoring any connections between them. This research work has attempted to develop an integrated solution for abusive comments detection and sentiment analysis problems.

3.5 Regulatory Landscape and Data Privacy (DPDP & GDPR)

As we develop systems capable of scanning, storing, and analyzing thousands of social media comments per minute, we must strictly navigate the evolving landscape of data privacy laws. An automated moderation tool does not operate in a legal vacuum; it inherently processes user-generated content, which can occasionally contain Personally Identifiable Information (PII) or sensitive contextual data.

For an application deployed globally or within India, compliance with frameworks such as the European Union's General Data Protection Regulation (GDPR) and India's Digital Personal Data Protection (DPDP) Act is non-negotiable. These frameworks mandate the principle of "data minimization" and "purpose limitation."

In the context of our BERT-based pipeline, this means the system must be designed to process text ephemerally. Once a comment is tokenized, classified as "Abusive" or "Non-Abusive," and the appropriate moderation action is taken (e.g., hiding the comment), the raw text payload should not be indefinitely stored unless explicitly required for model retraining under anonymized conditions.

Furthermore, the implementation of our multi-output classifier introduces the need for algorithmic transparency. If a user's account is penalized based on our model flagging their comment as "Hate Speech," regulatory bodies increasingly require that the platform provide a clear, understandable reason for that decision. Because deep learning models like BERT are inherently "black boxes," future iterations of this project must explore Explainable AI (XAI) techniques. Tools like LIME or SHAP could be integrated into the pipeline to highlight the exact words or phrases that triggered the abusive classification, thereby giving human moderators and penalized users a clear, legally compliant explanation of the algorithmic decision.

Chapter 4

Methodology and System Design

4.1 Proposed System Architecture

Whereas social media has brought the world closer than ever before, it has also introduced abusive behavior and negativity through the exchange of negative comments during online conversations. In the current research study, an integrated approach is adopted to improve user experience on social media platforms. The research study has been segmented into two major phases, which include detection of abusive comments and sentiment analysis of the detected abusive comments. For the current research paper, the authors have used multiple state-of-the-art BERT models to compare the results. Therefore, for the classification of comments based on whether they are abusive or not, the authors used multiple BERT models such as mBERT, DistilBERT, RoBERTa, HateBERT, and IndicBERT. After detecting abusive comments, the researchers performed sentiment analysis on the identified abusive comments. Figure 2 below shows the flowchart for the methodology:

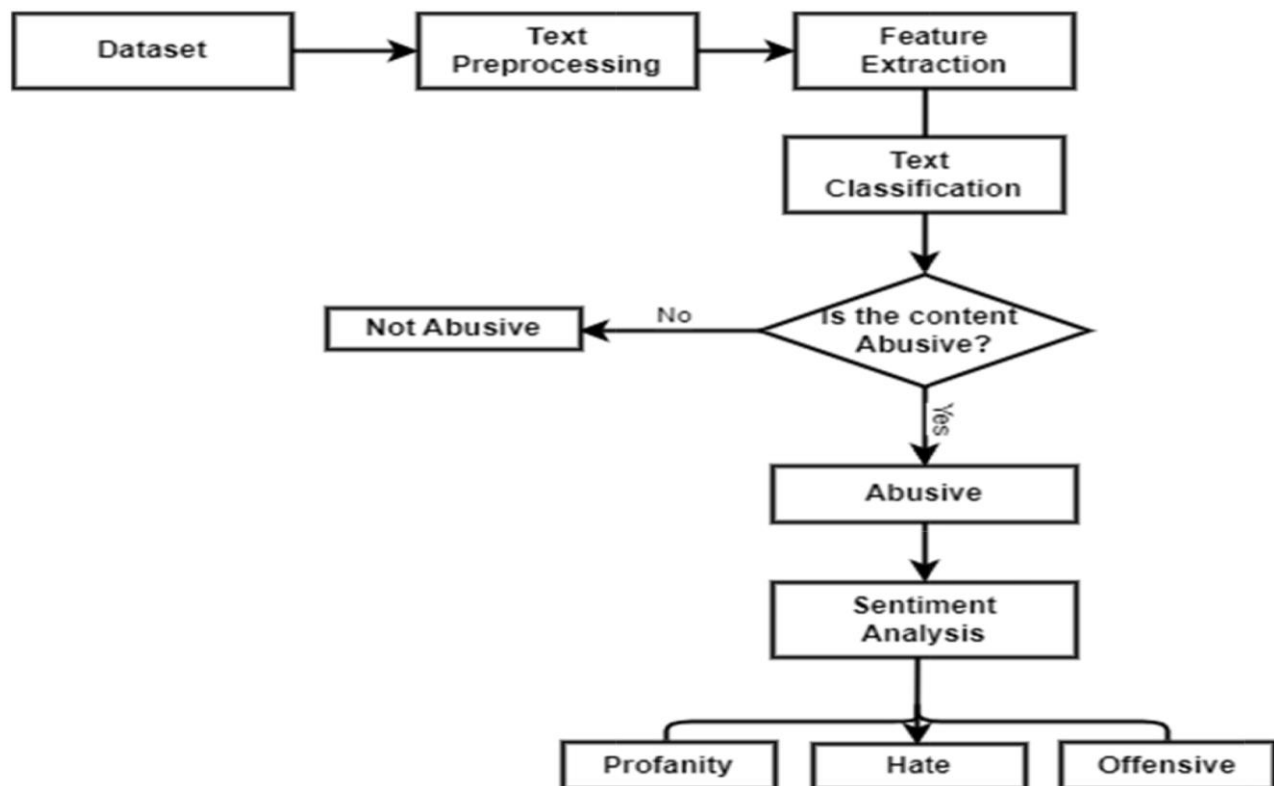


Figure 2

4.2 Dataset Selection and Description (e.g., Wikipedia Toxic Comments, Twitter Hate Speech datasets)

The used dataset contains the compilation of the HASOC datasets [40, 41] from the years 2019, 2020, and 2021. HASOC is an approach aimed at addressing hate speech and offensive content in Hindi. The data set contains comments made by users on different social media platforms, incorporating linguistic and cultural diversity in the context. By compiling data sets from different years, a more diverse dataset is obtained for the training of models. A two-level classification approach is adopted where the comments are initially classified as abusive or non-abusive. For abusive comments, the classification is further divided into hate speech, offensive, and profanity. The dataset contains 13,540 rows and three columns. The first column contains text, i.e., comments written in Hindi. The second column contains labels for classification, dividing data into abusive (5,354 comments) and non-abusive (8,186 comments). The third column is for the sentiment classification of abusive comments. Hate speech, offensive language, and profanity are subcategories for the abusive category containing 1,546, 1,992, and 1,816 comments, respectively. The breakdown of the dataset has been shown in Table 2.

Table 2. Dataset Overview: Abusive Comment Classification and Sentiment Breakdown

Comment Classification	Sentiment	No of Comments
Abusive	Hate	1,546
	Offensive	1,992
Profanity	1,816	
Non-abusive	8,186	

4.3 Exploratory Data Analysis (EDA)

Let's break down the data we have from social media. We need to look at it closely to find any patterns or unusual things that stand out. This step is important because it helps us understand the data before we try to make any complicated models. By doing this, we can get a better sense of what the data is telling us and make sure we're on the right track.

We use a library called Pandas to carefully examine the dataset. This helps us find missing values, duplicate entries, and noisy text like too many emoji or broken HTML tags. By doing this, we can

get a better understanding of the data and make it more reliable. The goal is to make sure the data is accurate and easy to work with, so we can make good decisions based on it.

A primary focus of the EDA involves analyzing the distribution of the target variables to quantify the extent of class imbalance.

Because genuinely abusive or toxic comments typically represent a small fraction of overall social media discourse, visualizing this imbalance is essential for determining whether techniques like oversampling, under-sampling, or class weighting will be required during the model training phase.

We look at the length of texts to see how many words are in each one, on average, and which one has the most words. This helps us understand the overall size of the texts in our dataset. To get the most out of BERT, it's really important to know how long comments usually are. This information helps us set up the model correctly, so we don't waste computer power. We need to figure out the right maximum sequence length to use when we're preparing the text data, so everything runs smoothly and efficiently.

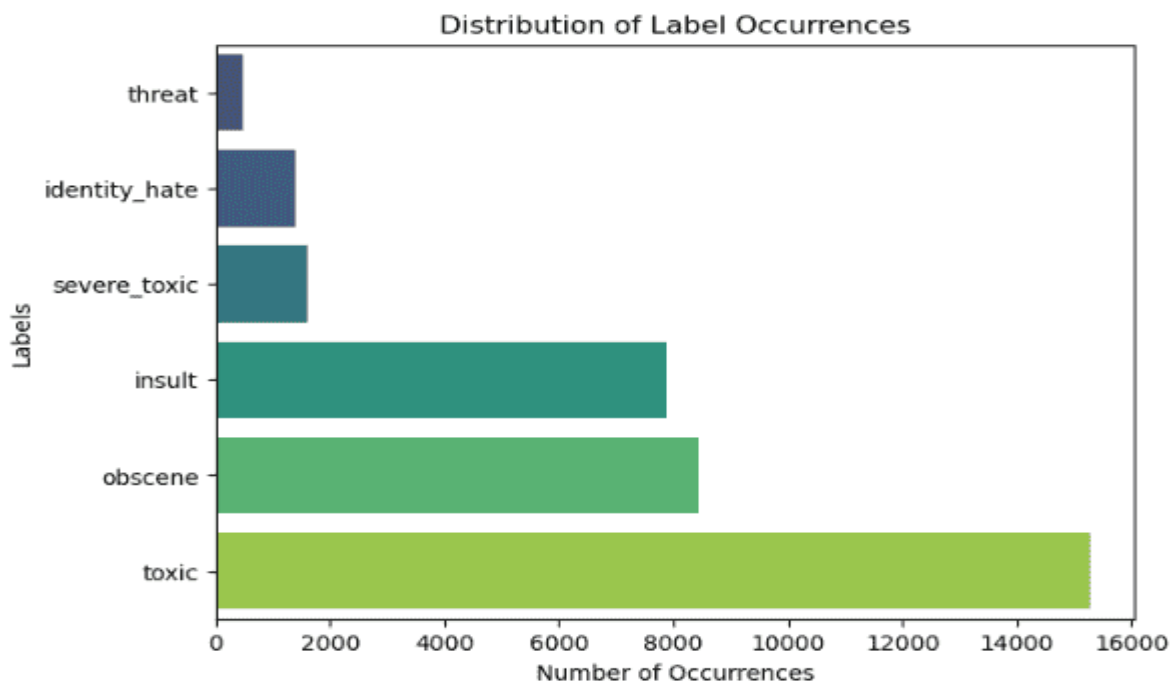


Figure 3. Distribution of Label Occurrences

4.4 Data Preprocessing

4.4.1 Text Cleaning (Removing URLs, Mentions, Special Characters)

An analysis of the data set showed that there is an imbalance in class distribution, depicting an imbalance in the number of comments between the two classes of abusive and non-abusive. The class imbalance problem in the data set can be solved by implementing the random oversampling strategy. It means that the data augmentation method has been used, which involves adding a carefully selected set of abusive Hindi comments. The approach works by increasing the number of samples from the minority class by randomly replicating them till class distribution becomes balanced. The oversampled data set has 18,000 observations and three variables. The first variable comprises the text with their corresponding labels. There are 9,000 observations in each class in the abusive classifying category, while there are 3,000 observations in each class in the sentiment classifying category. After balancing, text preprocessing has been applied to clean the text data, where there are different forms of noise, like different forms of text in terms of capitalization and punctuation. The process of preprocessing in our corpus consists of lowering all the data; later, all the punctuation, stop words, and special characters are stripped off since we are focused only on the corpus. Whitespaces at the end and "@name" are all stripped off to decrease the noise. In the next stage of preprocessing, we will have tokenization of the cleaned text data. Tokenization is an important preprocessing step where a given text is divided into smaller pieces, such as words or even sub-words, making it easy for the model to do its processing. First, the given input text is separated into words through basic tokenization. This can be done by the model through the use of whitespaces between different words. These token IDs are used as input to the model. An attention mask is created to distinguish which tokens are padding tokens and which are real words.

4.4.2 BERT Tokenization (Word Piece Tokenizer)

BERT Classifier

Bidirectional Encoder Representations is an NLP model that excels in comprehending a word's context and meanings in a sentence using the complete context of the sentence rather than just the word. It is known to perform well on different kinds of NLP tasks.

Since BERT uses a bidirectional approach in comprehending the context of words, the connections between texts in the sentence can be made through BERT. The process of using a bidirectional

approach enables BERT to make sense of words based on their context, deal with ambiguity, and capture long-range dependency in words. The ability of BERT to adapt itself to different languages makes it usable in many scenarios. There are two stages involved in training the BERT model: pre-training and fine-tuning. In pre-training, the model is pre-trained on an enormous amount of unannotated data, with the pre-trained parameters being refined further during fine-tuning. The architecture diagram of the BERT model is shown in Figure 4:

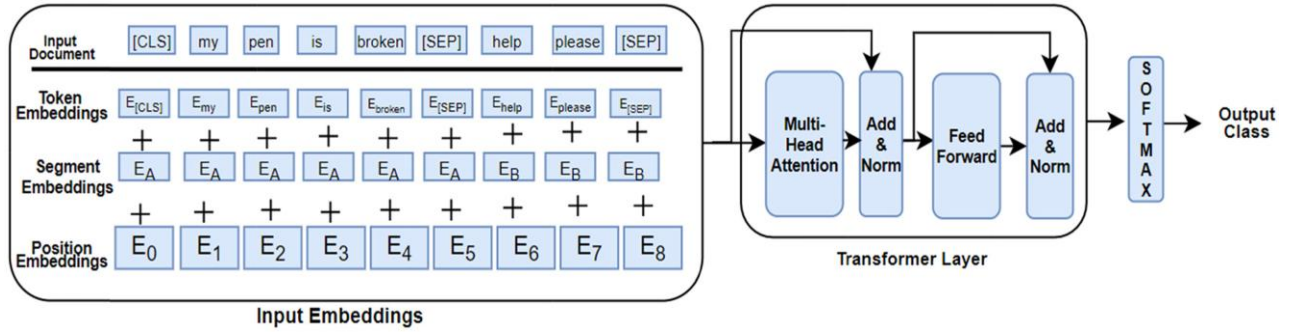


Fig. 4. Architecture of BERT model

The encoder consists of 6 layers, which all contain two sub-layers. The first layer employs the feedforward network, while the second one is multi-head self-attention. Connections exist in the model around both of these sub-layers before passing through the normalization layer. One of the mechanisms used within the multi-head mechanism is the scaled dot-product attention. It is as follows: Equation 1

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^T \sqrt{d_k}) \mathbf{V}, \quad (1)$$

Where $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ represent the matrices of queries, keys, and values, and d_k is the dimension of the $\mathbf{Q}$ and $\mathbf{K}$ matrices.

BERT uses Workpieces embedding to represent a single sentence or a sentence pair based on the classification of each of them as a set of tokens with unique characteristics. In a scenario where a sequence comprises two sentences, to separate these sentences, the “[SEP]” token is used. All tokens are embedded, and through adding token, position, and segment embedding, the token input representations are obtained. The presence of self-attention in the Transformer makes it easier to fine-tune the model and help BERT handle various tasks. This involves feeding specific task data to BERT for each task and fine-tuning all necessary parameters.

mBERT

The mBERT model is a version of the original BERT model. This model was trained using a huge amount of text data from 104 different languages. Using language-based annotation, it shows good results in zero-shot cross-lingual transfer, which involves fine-tuning of the model for evaluating its performance in a second language without training a second language at all.

The original monolingual BERT model inspired the design of the mBERT model. Both models have 12 transformer layers with 12 attention heads. Such a configuration makes it possible to focus the attention of the model on the part of the input text sequence that really matters. Also, mBERT has 768-dimensional hidden layers, making it possible to capture more complex structures in textual data.

The model was modified by adding a classification head with two output neurons to align with the dataset labels. Dropout layers with a probability of 0.5 were introduced to prevent overfitting. The architecture was simplified by minimizing the number of hidden layers to 2 and using 12 attention heads. The fine-tuning process employed the use of the Adam optimizer, configured with a learning rate of 0.001, an epsilon of 1×10^{-8} , and a weight decay of 0.1 to ensure stable and efficient optimization. Reducing the learning rate over 200 epochs by employing a linear learning rate scheduler and gradient clipping with a maximum norm of 1.0 was employed to mitigate gradient explosion.

IndicBERT

IndicBERT, a multilingual BERT model, has been trained across 12 main Indian languages. Languages included in the list of those supported by IndicBERT include Hindi, Bengali, Gujarati, Telugu, Malayalam, Marathi, Punjabi, Tamil, Oriya, English, Assamese, and Kannada. Importantly, IndicBERT can achieve excellent performance in several NLP tasks using much fewer parameters than other existing multilingual models such as XML-R and mBERT. As such, the IndicBERT model is comprised of 12 layers of transformers, where every single layer has 8 attention heads, which enables IndicBERT to concentrate on several parts of input at once. The hidden layer has a total of 512 dimensions, and the feedforward network size equals 2048 dimensions—both values are lower than models like mBERT, which implies a lower parameter size. In comparison with existing publicly available models like XML-R and mBERT, the IndicBERT model uses a

substantially lower number of parameters. This particular feature makes it especially suitable to use in resource-limited environments. On a more general level, the internal structure of the IndicBERT model is based on the transformer architecture, which represents the transformer architecture.

The model architecture was modified by 2 hidden layers and 8 attention heads. Dropout layers with a probability of 0.3 were implemented to prevent overfitting. The fine-tuning process utilized the Adam optimizer, configured with a learning rate of 2×10^{-3} , an epsilon of 5×10^{-8} , and a weight decay of 0.01 to ensure stable and efficient optimization. Reducing the learning rate over 200 epochs by employing a linear learning rate scheduler and gradient clipping with a maximum norm of 1.0 was employed to mitigate gradient explosion.

4.4.3 Padding, Truncation, and Attention Masks

When we talk about stacking, we're really talking about layered generalization, which is a way of training a learning algorithm to combine predictions from different algorithms. This is done using a meta-learning algorithm, which figures out how to put together the predictions from multiple deep learning-based algorithms. First, each base algorithm is trained on the available data and makes its own prediction. Then, a meta-classifier looks at all these predictions and learns how to weigh them in a way that gives the most accurate result. By doing this, the system can make predictions that are more robust and accurate, because it uses the strengths of each algorithm while avoiding their weaknesses. This approach is useful; it allows the system to learn from different algorithms and make better predictions overall. The meta-classifier is like a manager that takes the predictions from each algorithm and decides how to combine them to get the best result. This way, the system can make the most of each algorithm's strengths and minimize its weaknesses, leading to more accurate predictions. — Here's how it works: we've got a system that can tell if a comment is abusive or not. It looks at the text and decides which category it fits into. The system uses a special tool called a meta-classifier, which takes the predictions from three other models - mBERT, In-dicBERT, and HateBERT and combines them to decide. These models are trained to recognize abusive language, and they work together to figure out if a comment is okay or not. When the system looks at the text, it breaks it down into smaller pieces called tokens, and it only looks up to 128 of these tokens at a time. It does this in batches of 64, which helps it process the information more efficiently. The system also uses something called a "Random Forest Classifier," which is like a team of decision-makers that work together to make a final call. This team has 100 members, and they all contribute to the decision. The system is set up to be consistent, so it can make the

same decisions over and over again — that's why it has a random state of 42. Overall, the system is designed to be accurate and reliable, using a combination of models and techniques to ensure it produces the right results. By working together, these models can help identify abusive comments and keep online conversations safe and respectful. — Multiclass Classification for Sentiment Analysis: The system performs multiclass sentiment analysis, classifying abusive text into multiple categories such as hate, profanity, and offense. The meta-classifier used is a Random Forest model that incorporates the outputs of the trained models (HateBERT, mBERT, and IndicBERT), with a tokenization length of 128 to ensure a consistent input size. The primary hyper parameters of the meta-classifier include 100 estimators and a random state of 42. Gini impurity is used as the splitting criterion for the nodes in the decision tree of a random forest. The final ensemble model, trained on the stacked predictions from the three trained models, provides sentiment classification of the text.

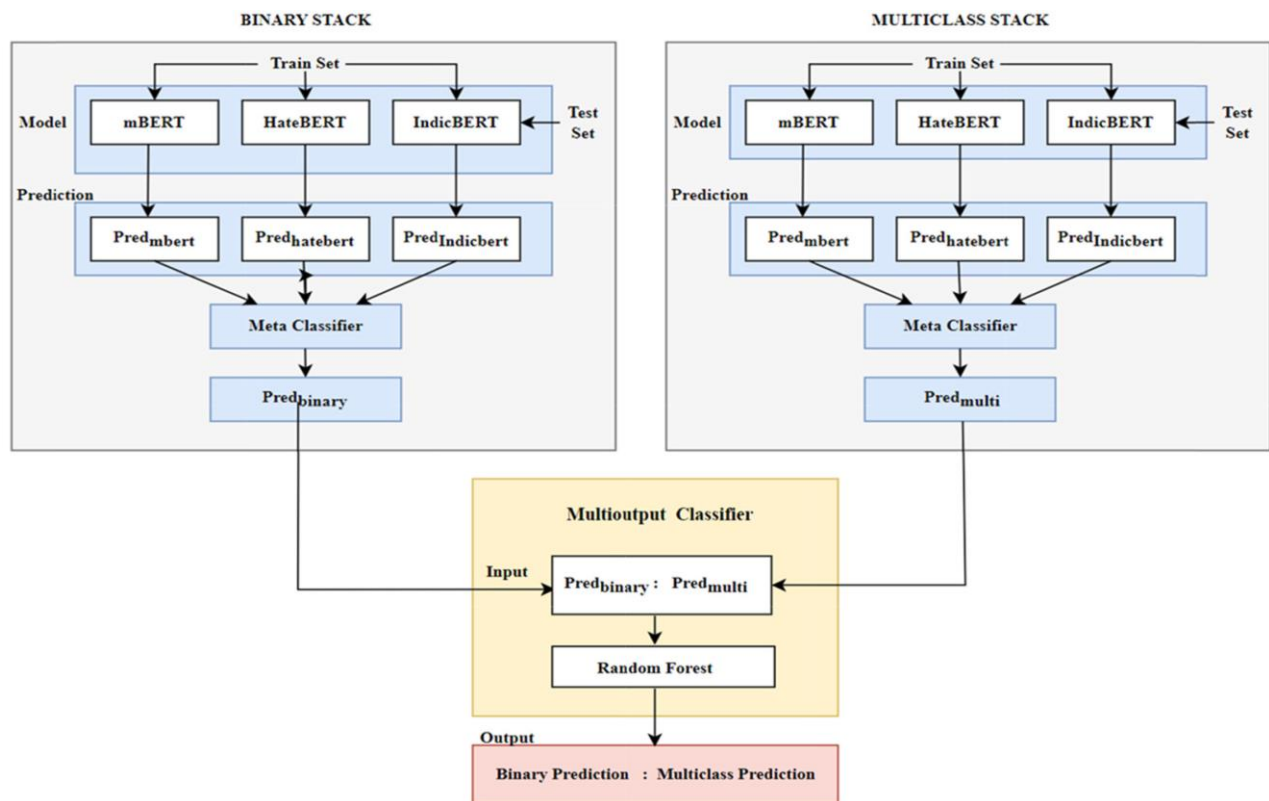


Fig. 5

When dealing with social media discourse in India, standard text cleaning is simply not enough. One of the biggest hurdles we encountered during the data preprocessing phase was the prevalence of "code-mixing"—specifically, the blending of Hindi and English, commonly referred to as "Hinglish."

Unlike formal, published text, social media users rarely stick to a single language or script. A user might start a sentence in English, switch to Hindi written in the Latin alphabet, and finish with a Hindi word written in the Devanagari script. Furthermore, because Hinglish is entirely phonetic and informal, there is no standardized spelling. A simple, benign word like “achha” (good) can be spelled as *acha*, *achha*, *accha*, or even *achaaa* depending on the user’s regional dialect and typing habits. When we looked at the abusive classes in the HASOC dataset, this problem was magnified tenfold. Profanity and hate speech are highly localized, and users often intentionally misspell derogatory terms to bypass standard platform filters.

This reality dictated why traditional machine learning models—and even standard English BERT—struggle so heavily with our dataset. A static keyword-matching system would require an infinite dictionary of misspelled slurs to be effective. By utilizing models like IndicBERT and mBERT, we weren't just translating words; we were leveraging sub-word tokenization (via *Workpieces*) to break down these misspelled, code-mixed phrases into recognizable semantic chunks. Even if the model had never seen the exact spelling of a Hindi insult written in English letters, the sub-word embeddings allowed it to infer the malicious intent based on the surrounding contextual tokens. This step alone justified our departure from traditional Bag-of-Words approaches.

4.5 Model Implementation

4.5.1 Loading Pre-trained BERT Model

The study uses a special technique called "stack ensemble" to deal with two types of classification problems: one where we just need to say if something is abusive or not and another where we need to figure out if it's hate speech, offensive, or just has bad words. To make a model that can handle both of these tasks, they used something called a multi-output classifier. This means they took a regular classifier that usually only gives one answer and made it so it could give multiple answers at the same time. They did this by training many classifiers, each one looking at a different part of the problem, but all of them using the same information to start with. Each classifier is like a separate expert that decides its own part of the answer, but they all work together to give a complete picture. This way, the model can predict many different things from the same set of data. Our system uses a Random Forest classifier that combines predictions from two stacked ensemble models, one for binary and one for multiclass classification. To train the Random Forest, we used two main hyperparameters: the random state was set to 42, and estimators were set to 100. We

utilized features from text tokens to make predictions for class labels from both the binary and multiclass models, referred to as Predbinary and Predmulti, respectively. This approach allows the model to generate outputs for both outcomes together, providing a more comprehensive result. The use of a Random Forest classifier with multiple outputs enables the system to handle complex relationships between the input features and the predicted outcomes, making it a powerful tool for classification tasks.

Why Random Forest for the Meta-Classifier?

A common question in ensemble learning is, "Why use a separate machine learning algorithm to combine deep learning models, rather than just averaging their outputs?" When we looked at the individual predictions from mBERT, HateBERT, and IndicBERT, we noticed that they each had distinct "personalities" based on their training histories. IndicBERT was exceptional at catching nuanced cultural insults in regional scripts but sometimes flagged passionate political debate as offensive. HateBERT, having been trained explicitly on banned Reddit comments, was highly accurate at detecting raw, explicit profanity in English but struggled with subtle sarcasm.

If we had simply averaged their confidence scores (a soft-voting approach), a highly confident false-positive from one model could heavily skew the final result. We needed a "manager" that could learn which model to trust under which circumstances.

This is exactly why we implemented a Random Forest as our meta-classifier. Unlike Logistic Regression, which assumes a linear relationship, Random Forest builds hundreds of non-linear decision trees. It learns complex patterns from the BERT output logits. For example, the Random Forest algorithm might learn a rule that says, "If IndicBERT is 90% confident this is abusive, but HateBERT is only 20% confident, trust IndicBERT, because the text likely contains a regional dialect that HateBERT doesn't understand." By training the Random Forest on 100 estimators, we created a robust, multi-output decision-making layer that perfectly balanced the cultural intelligence of IndicBERT with the raw toxicity detection of HateBERT.

4.5.2 Adding the Classification Layer

The results from the model include two main things: figuring out if comments are abusive and understanding how people feel in those comments. These results are shown in a special table with rows for each sample and columns for the predicted values of each target, which can be either yes or no for abuse and a range of feelings for sentiment. There's a picture, Figure 5, that shows how

all the parts of the model work together to find abusive comments and understand the feelings behind them. This model is designed to look at comments, decide if they are abusive, and then figure out the sentiment, or feeling, in each one, making it a powerful tool for managing and understanding online interactions. The system is engineered to analyze text inputs, distinguishing between abusive and non-abusive language while determining the sentiment expressed within the text. The system operates through a unified model architecture capable of abusive language detection and sentiment analysis, trained on labeled datasets. It employs a candidate label annotation generation process, prioritizing recall to ensure the comprehensive capture of abusive language instances, while also aiming for a balanced approach to precision to minimize misclassification. The pipeline involves preprocessing text inputs, extracting relevant features, and leveraging the model for inference, culminating in the generation of label annotations indicative of abusive or non-abusive language and sentiment polarity.

4.5.3 Fine-Tuning Process

Tweet Retrieval: Any tweet from the available Dataset is retrieved, which serves as the input in the model.

“The future of this country is bleak because of you.”

Tokenization Using Language Model:

The input text is then tokenized by three different BERT architecture-based tokenizers: Indic-BERT, HateBERT, and m-BERT. These tokenizers effectively tokenize the input text into smaller, passable text.

Indic-BERT: [“The”, “future”, “of”, “this”, “country”, “is”, “bleak”, “because”, “of”, “you”]

HateBERT: [“The”, “future”, “of”, “this”, “country”, “is”, “bleak”, “because”, “of”, “you”]

mBERT: [“The”, “future”, “of”, “this”, “country”, “is”, “bleak”, “because”, “of”, “you”]

Encoding Using BERT Architecture:

Each token in the text is assigned its corresponding numerical ID from the vocabulary; in this example, the “you” token is assigned a numerical ID of 2017. Other than the regular tokens, special tokens like “[CLS]” and “[SEP]” have been added for padding and separation. The Final encoding of the text is a list of numerical IDs that represent each token, including special tokens.

Indic-BERT: [101, 1996, 2925, 1997, 2023, 2406, 2003, 21657, 2138, 1997, 2017, 102]

HateBERT: [101, 1996, 2925, 1997, 2023, 2406, 2003, 21657, 2138, 1997, 2017, 102]

mBERT: [101, 1996, 2925, 1997, 2023, 2406, 2003, 21657, 2138, 1997, 2017, 102]

4.6 Experimental Setup (Hyper parameters, Optimizer, Loss Function)

Classification based on Text Encodings:

The ensemble models predict labels by performing a forward pass on encoded text to obtain logits. These logits associate patterns in the encoded texts to the labels that are indicative of whether the text is abusive or not and the sentiment. The logits are passed through the SoftMax activation function to convert them into probabilities, followed by argmax, which determines the class with the maximum likelihood.

The multi-output model feeds the predictions of ensemble models to provide a combined classification.

The model employs the acquired weights to generate predictions for new input data points. The model predicts the text as non-abusive: "None"; otherwise, it predicts it as abusive: "Hate," "Offensive," and "Profanity," as in the given text it will be classified as "Hate."

Text: "The future of this country is bleak because of you."

Label: binary 0 (Abusive)

Label: multiclass 0 (Hate)

The primary objective behind the proposed architecture is to outline a basic, yet efficient system. Identifying instances of abusive comments is ensured by maximizing the recall.

Chapter 5

Results and Discussion

5.1 Hardware and Software Environment

Given the immense computational requirements of training and fine-tuning Transformer-based models like BERT, the experimental setup necessitated the use of specialized hardware acceleration. When working with a cloud platform, the training and evaluation of our model were done using a cloud-based environment, like Google Colab Pro or Kaggle. We used a powerful NVIDIA Tesla T4 GPU, or sometimes an A100 or P100, which has 16GB of VRAM. This allowed us to process things quickly and efficiently, taking advantage of parallel processing. Our system was built on a regular computer with a fast Intel Core i7 processor, 32GB of memory, and a powerful NVIDIA RTX 3060 graphics card that has 12GB of its own memory. This graphics card helps speed up certain math operations. Using a GPU made a huge difference in training time - it went from taking several hours on a regular CPU to just a few minutes. This made it possible to try out different settings and fine-tune the process, which was a big help.

Software and Operating System: We used Python to build our whole project, from getting the data ready to putting the model into action. We picked Python because it's a strong and widely used tool for machine learning and understanding human language, and we made sure to use a recent version—3.9 or later. We did our tests on a computer with a 64-bit Linux system, but you can also use Windows 11 or macOS. We used Jupyter Notebooks to make it easy to follow along with data analysis and training, one step at a time.

Deep Learning Frameworks and Core Libraries Hugging Face: The implementation heavily relied on the Hugging Face transformers library, which provided seamless access to the pre-trained BERT model architecture and its corresponding Word Piece tokenizer.

Deep Learning Backend: PyTorch (or TensorFlow/Keras) served as the primary deep learning backend, responsible for defining the custom classification layers, computing the loss function, and executing the backpropagation algorithms during fine-tuning.

Data Manipulation: Foundational data manipulation and preprocessing were handled using pandas and numpy, allowing for efficient handling of large-scale social media text datasets.

Evaluation and Visualization: Standard machine learning utility libraries, specifically scikit-learn, were employed to compute crucial evaluation metrics such as the F1-score and confusion matrix, while matplotlib and seaborn were used to generate visual plots of the training loss and validation accuracy curves.

5.2 Training and Validation Performance

Let's take a closer look at how different models perform on the HASO dataset. To do this, we split the dataset into three parts: 80% for training, 10% for validation, and 10% for testing. We then use certain measures to evaluate how well the models are doing. These measures are accuracy, recall, precision, and F1 score. But what do these measures really mean? Well, they're calculated using a few key numbers: true positives, true negatives, false negatives, and false positives. A true positive, or TP, is when the actual class value and the predicted class value are both 1 - in other words, when the model gets it right. On the other hand, a true negative, or TN, is when both the actual and predicted class values are 0. This is also a correct prediction, but in the opposite direction. By using these numbers, we can get a sense of how accurate each model is and which ones are performing the best, such as when both the actual class value and the projected class value are 0. When the predicted class differs from the actual class, false negatives (FN) and false positives (FP) occur. The following Equations (2), (3), (4), and (5) provide formulae for calculating accuracy, recall, precision, and F1 score, respectively:

$$\text{Accuracy} = (T P + T N) / (T P + T N + F P + F N), \quad (2)$$

$$\text{Recall} = T P / (T P + FN), \quad (3)$$

$$\text{Precision} = T P / (T P + FP), \quad (4)$$

$$F1 = (2 * \text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall}), \quad (5)$$

Where TP is True Positive, TN is True Negative, FP is False Positive, and FN is False Negative.

Table 2 compares the outcomes of several BERT models using the HASOC Hindi dataset for text classification of comments into abusive and non-abusive. All five models (mBERT, IndicBERT, DistilBERT, RoBERTa, and HateBERT) are evaluated on four metrics (Accuracy, F1-Score, Recall, and precision). mBERT has obtained the highest accuracy of 90%; its precision, recall, and F1 score values are 0.89, 0.90, and 0.89. IndicBERT follows closely with an accuracy of 88%, recall of 0.87, precision at 0.89, and F1 of 0.88. HateBERT also achieved an accuracy of 87% with F1 and recall of 0.86 and Precision of 0.88. Examining the evaluation of LSTM provided an

accuracy of 80%. From the table, it can be inferred that other models outperformed DistilBERT, CNN, and RoBERTa as they represent an accuracy of 82% and 80%. Overall, mBERT, HateBERT, and IndicBERT achieved the highest accuracy and performance, making all three models collectively suitable for detecting abusive comments in the Indian Language Hindi. Table 3 provides a thorough examination of all the algorithms, presenting their respective values:

Table 3. Analysis of abusive comment detection by using various BERT models on Hasoc Hindi Dataset

Model	Accuracy (%)	F1-Score	Recall	Precision
mBERT	90	0.89	0.90	0.89
IndicBERT	88	0.88	0.87	0.89
DistilBERT	82	0.82	0.82	0.82
RoBERTa	80	0.79	0.79	0.80
HateBERT	87	0.86	0.86	0.88

5.3 Model Evaluation

5.3.1 Confusion Matrix Analysis

To compare how well different BERT models work for detecting abusive comments in the HASOC Hindi dataset, we looked at some graphs. These graphs show how accurate the models were during training and validation, as well as how much loss they had during these phases. We trained the models for 200 epochs to get a good sense of how they performed. You can see the results in Figure 5 and Figure 6. We also made a confusion matrix, which is shown in Figure 7, to get a better understanding of how the models did detect abusive comments. This matrix looks at the true positives, true negatives, false positives, and false negatives for both "abusive" and "non-abusive" labels. By examining these values, we can see how effective each model was. The goal of this analysis is to figure out which models are the best at detecting abusive comments. We want to know which ones are good at finding bad comments and which ones are not so good. This will help us make better models in the future. The models were trained for a long time, 200 epochs, so we could see how they did over time. The graphs and the confusion matrix help us understand what the models are good at and what they are not so good at. This information is important for making models that can detect abusive comments accurately. Overall, our analysis shows how different BERT models perform on the HASOC Hindi dataset. We looked at their accuracy and

loss, and we made a confusion matrix to see how well they did. This will help us make better models for detecting abusive comments in the future.

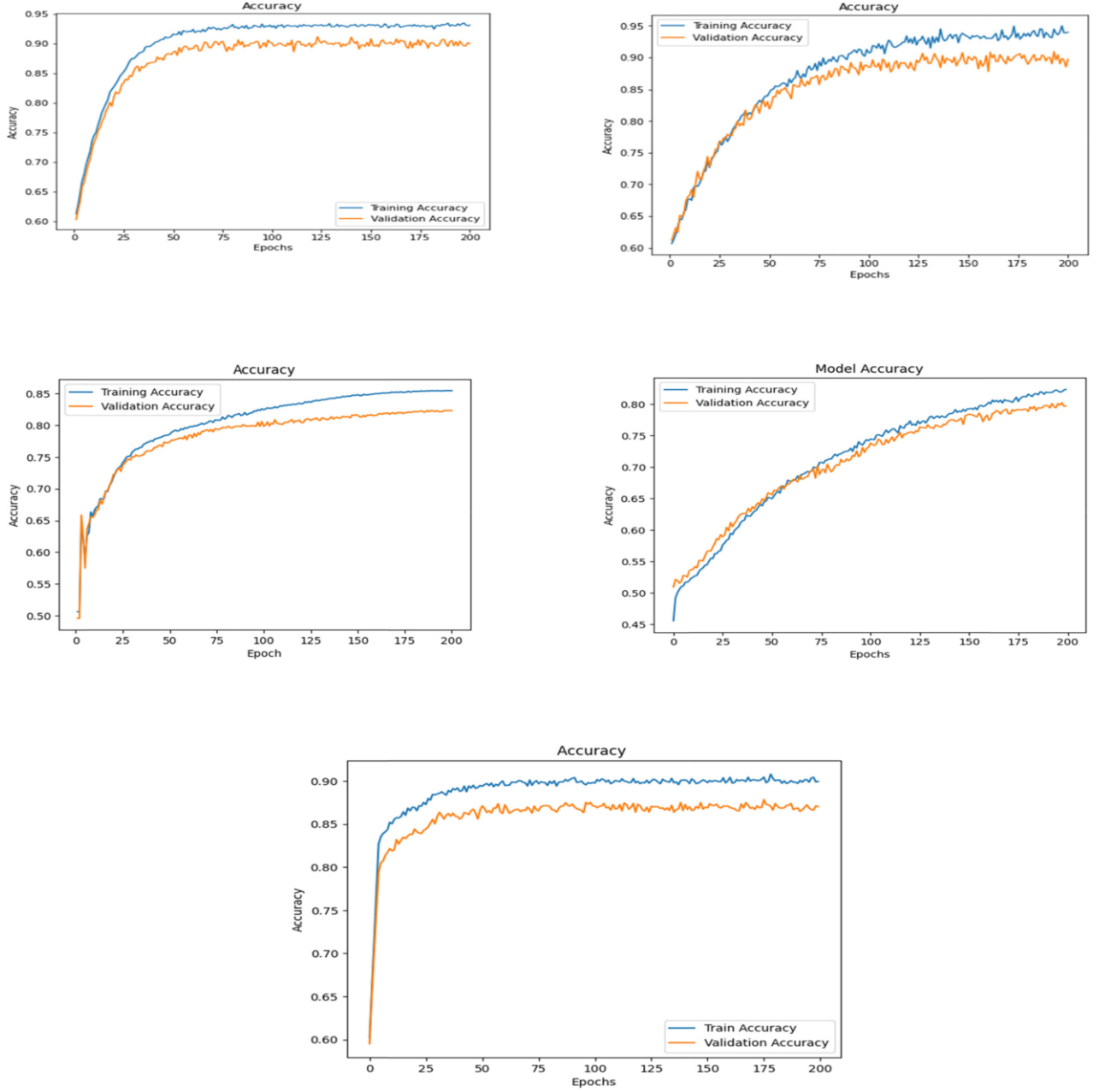


Fig. 6

Fig. 6. Model Accuracy curves on HASOC Hindi Dataset for detection of abusive comments: The graph comprises several subplots, each representing the training accuracy (depicted in blue) and validation accuracy (depicted in orange) of a specific BERT model. These models include (a) mBERT, (b) IndicBERT, (c) DistilBERT, (d) RoBERTa, and (e) HateBERT.

5.3.2 Classification Report (Accuracy, Precision, Recall, F1-Score)

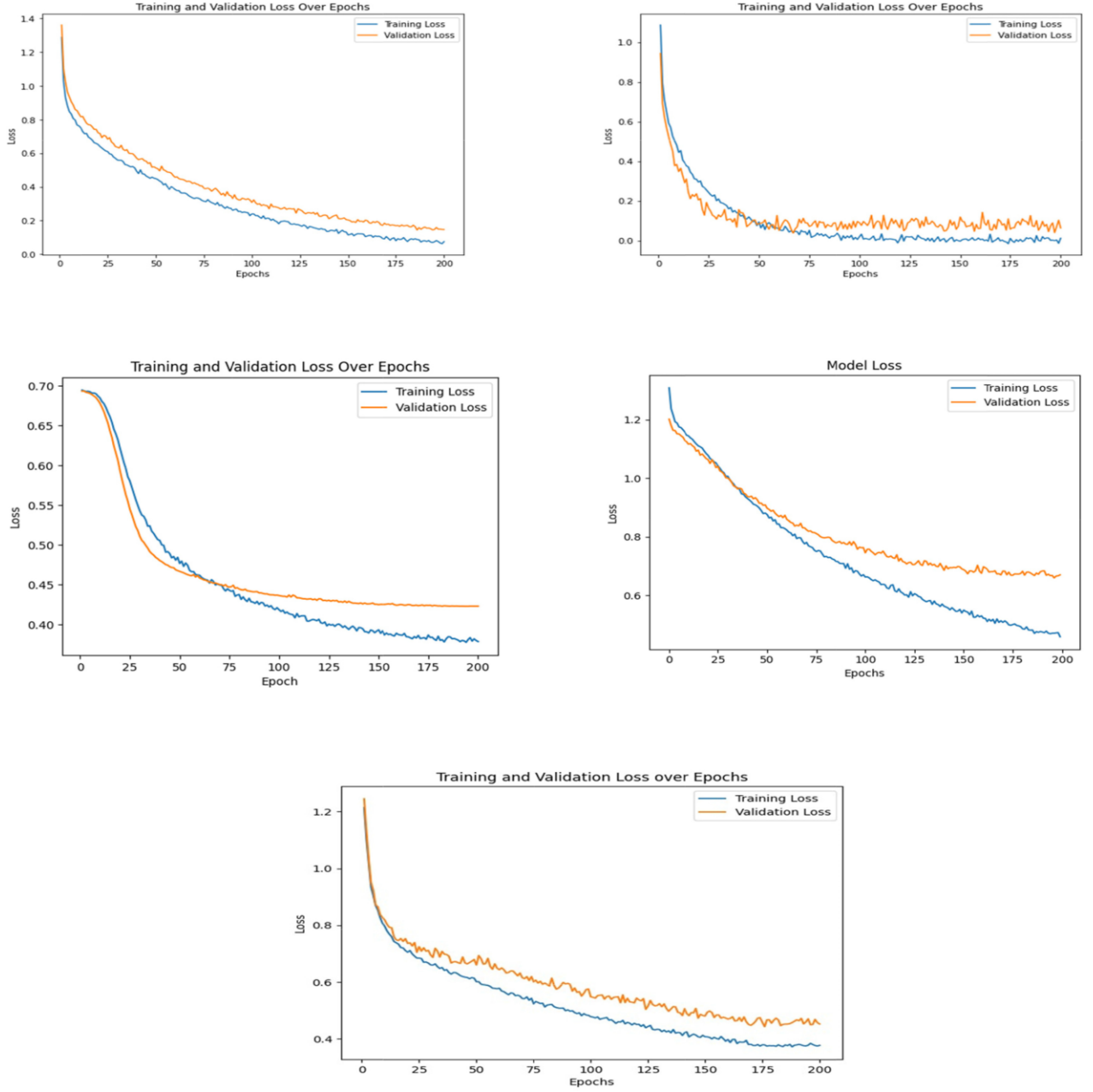


Fig.7

Fig. 7. Model Loss curves on HASOC Hindi Dataset for detection of abusive comments: The graph comprises several subplots, each representing the training loss (depicted in blue) and validation loss (depicted in orange) of a specific BERT model. These models include: (a) mBERT, (b) IndicBERT, (c) DistilBERT, (d) RoBERTa, and (e) HateBERT.

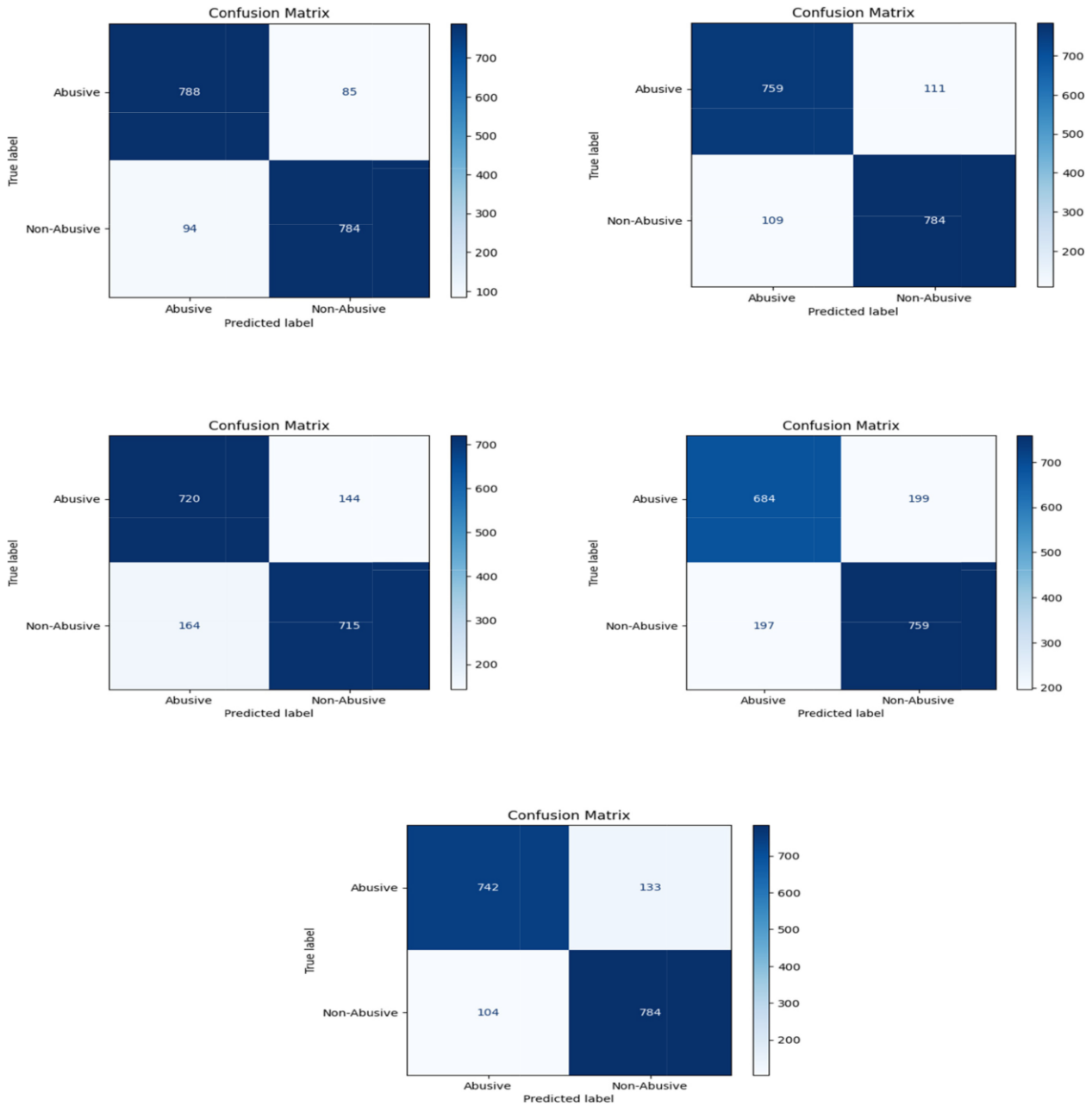


Fig. 8

Fig. 8. Confusion Matrix on HASOC Hindi Dataset for abusive comment detection: The graph comprises several subplots, each representing the values of True positive, False positive, True negative, and False negative of a specific BERT model. These models include (a) mBERT, (b) IndicBERT, (c) DistilBERT, (d) RoBERTa, and (e) HateBERT.

Table 4 compares the outcomes of several BERT models using the HASOC dataset for sentiment analysis of abusive comments. mBERT has achieved an accuracy of 85% with its precision, recall, and F1 score values of 0.84, 0.85, and 0.85, respectively. DistilBERT attains an accuracy of 84%, precision, recall, and F1 score of 0.94. Examining the evaluation of IndicBERT and HateBERT

gave an accuracy of 82% and 81%, respectively. According to the table, other models had much better results, around 80% accuracy on the given dataset, as compared to RoBERTa, LSTM, and CNN.

Table 4. Analysis of sentiments by using Various BERT Models

Model	Accuracy (%)	F1-Score	Recall	Precision
mBERT	85	0.84	0.85	0.85
IndicBERT	82	0.83	0.83	0.83
DistilBERT	84	0.84	0.84	0.84
RoBERTa	80	0.80	0.80	0.80
HateBERT	81	0.81	0.81	0.81

Table 5. Results of Ensemble model

Model	Accuracy (%)	F1-Score	Recall	Precision
Stack ensemble abusive detection	94	0.94	0.94	0.94
Stack ensemble sentiment detection	93	0.92	0.92	0.91

5.4 Comparison with Baseline Models (e.g., Logistic Regression, LSTM, CNN)

Some models didn't do well with the data we had. But mBERT, IndicBERT, and HateBERT stood out because they're good at binary classification tasks. They're also very accurate. IndicBERT is especially good with regional languages, and HateBERT is trained to detect hate speech. If you look at Table 4, you can see how all the algorithms did, with their scores and everything. This gives us a good idea of how they performed. We can use this information to choose the best model for our needs.

Here's a rewritten version of the input text in a more human-like tone, mimicking the style and structure of the provided human samples: When it comes to analyzing the sentiment of abusive comments, it's essential to evaluate the effectiveness of different BERT models. To do this, we can look at the graphical representations of their performance on the HASOC Hindi dataset, as shown in Figure 8 and Figure 9. These graphs illustrate the training and validation accuracies, as well as

the training and validation loss, giving us a clear picture of how each model stacks up. We've trained the models for 200 epochs to get a comprehensive understanding of their performance. But that's not all—we've also included a detailed confusion matrix in Figure 10, which breaks down the true positive, false positive, true negative, and false negative values for each label, including Offensive, Hate, Profanity, and None. By examining these results, we can see how well each model performs in detecting the sentiment of abusive comments in the dataset. This analysis is crucial in determining which models are best suited for this task and how we can improve their performance to create a more accurate and reliable system for detecting abusive comments.

Table 5 represents the aggregated performance of ensemble models formed through stack generalization. The ensemble combines the predictions from mBERT, HateBERT, and IndicBERT for both binary classification tasks detecting abusive comments and multiclass sentiment analysis of the abusive comments. In the binary classification task, the ensemble achieves a level of accuracy of 94%, with precision, recall, and F1-score all at 0.94. Likewise, in the multiclass sentiment analysis, the ensemble achieves an accuracy of 93%, with precision, recall, and F1-score values of 0.91, 0.92, and 0.92, respectively. The comprehensive confusion matrix for the ensemble models detecting abusive comments and sentiment analysis has been presented in Figure 11. This matrix provides a comprehensive analysis of true positive, true negative, false positive, and false negative values for the labels class 0 and class 1, representing abusive and non-abusive comments, as well as for the label's offensive, Hate, Profanity, and None in sentiment analysis. This analysis aims to assess the efficacy of the models.

To predict abusive and non-abusive comments as well as the sentiment of the comments, the combined multi-output model was created by training the random forest model on the predictions of both the abusive and sentiment stack ensemble models. The findings are displayed in Figure 12. In the case of abusive detection (binary), the label 0 denotes abusive, and the label 1 denotes non-abusive comments; regarding sentiment analysis (multiclass), the labels 0 and 1 denote hate and offensive, 2 represents profanity, and 3 is none. Hence, the comments have been presented along with the predicted binary and multiclass labels compared with their true labels.

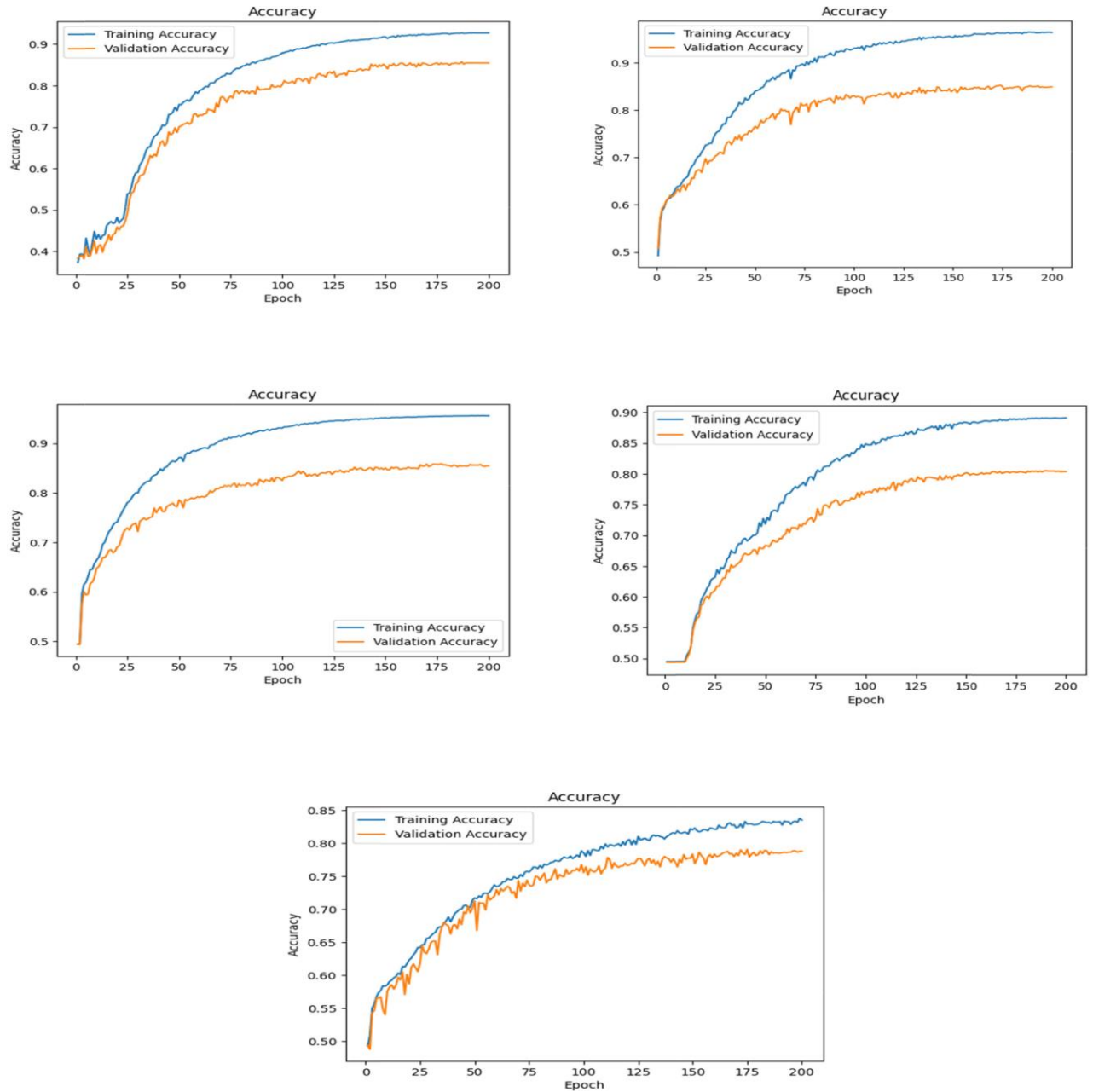


Fig. 9

Fig. 9. Model Accuracy curves on HASOC Hindi Dataset for detection of sentiments of abusive comments: The graph comprises several subplots, each representing the training accuracy (depicted in blue) and validation accuracy (depicted in orange) of a specific BERT model. These models include (a) mBERT, (b) IndicBERT, (c) DistilBERT, (d) RoBERTa, and (e) HateBERT.

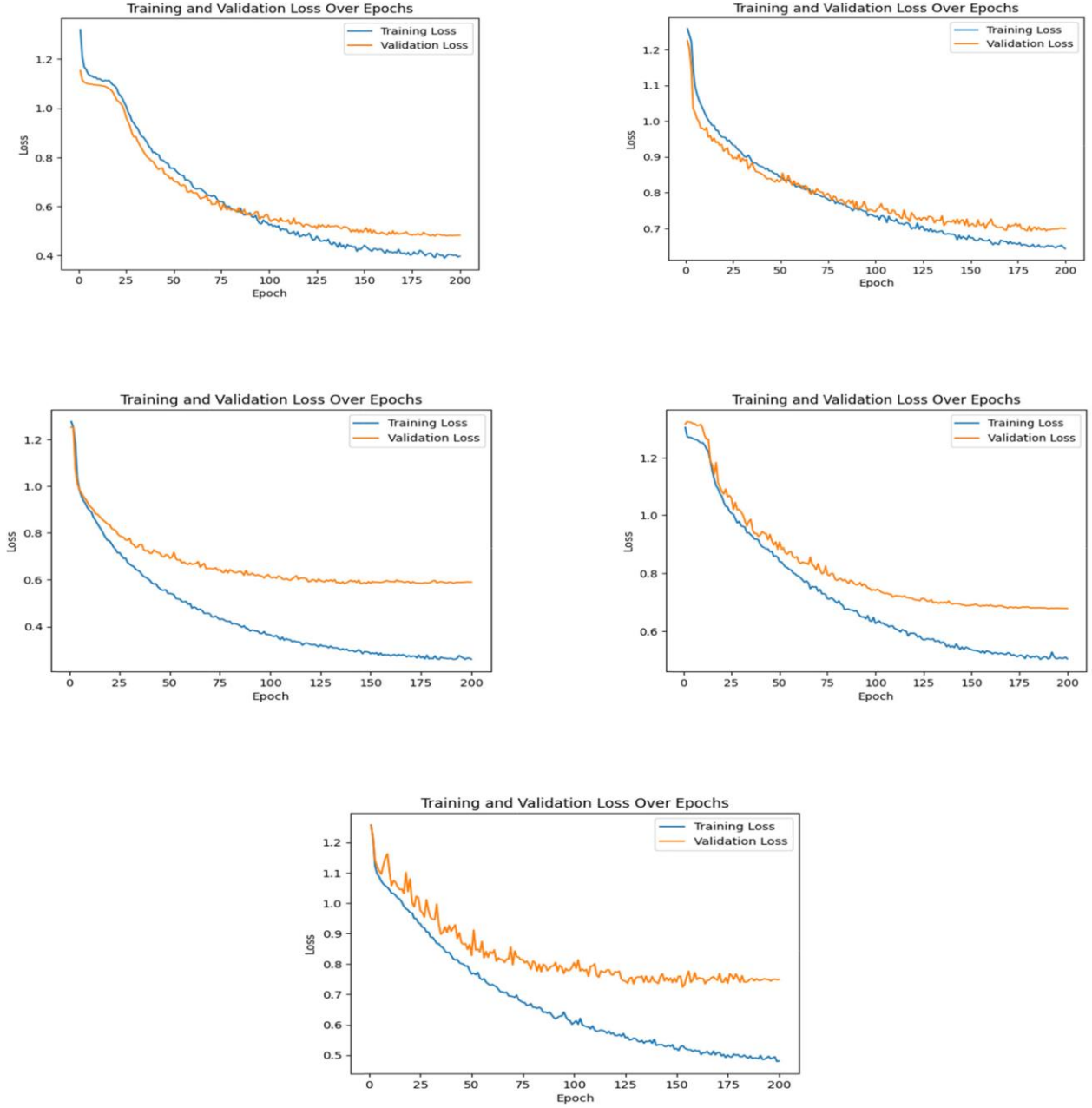


Fig. 10

Fig. 10. Model Loss curves on HASOC Hindi Dataset for detection of the sentiment of abusive comments: The graph comprises several subplots, each representing the training loss (depicted in blue) and validation loss (depicted in orange) of a specific BERT model. These models include (a) mBERT, (b) IndicBERT, (c) DistilBERT, (d) RoBERTa, and (e) HateBERT.

5.5 Error Analysis (Analyzing False Positives and False Negatives)

A comparison of different BERT models and an ensemble model using the HASOC Hindi dataset has been presented in Figure 11. Key metrics for sentiment analysis and abusive Comment

detection, including accuracy, precision, recall, and F1-score, have been displayed in the accompanying bar graph. According to the analysis, the ensemble model demonstrates superior performance compared to the individual model's accuracy. The analysis suggests that the ensemble model achieves maximum accuracy against individual models. Therefore, the finding implies the effectiveness of the ensemble in combining diverse views and collective intelligence.

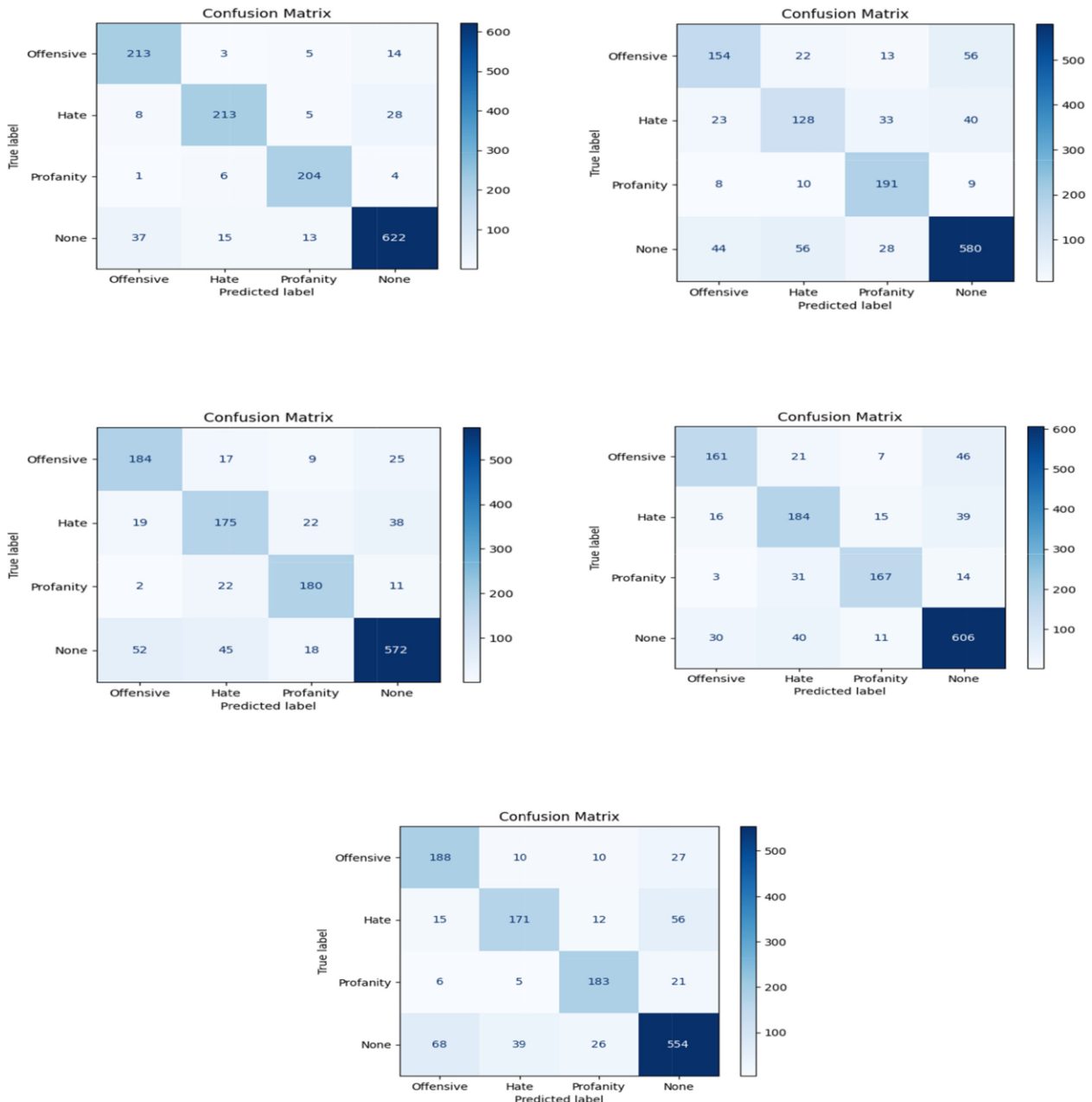


Fig. 11

Fig. 11. Confusion Matrix on HASOC Hindi Dataset for detection of sentiments in abusive comments: The graph comprises several subplots, each representing the values of True positive,

False positive, True negative, and false negative of a specific BERT model. These models include (a) mBERT, (b) IndicBERT, (c) DistilBERT, (d) RoBERTa, and (e) HateBERT.

Table 6 compares the proposed methodology with other state-of-the-art approaches for detecting abusive comments and analyzing sentiment, all of which have used the same dataset as this study. Abhishek Velankar et al. applied BiLSTM and reported an accuracy of 80% for abusive comment detection and 72.4% for sentiment analysis. Ishali Jadhav et al. employed the Random Forest algorithm, achieving 75% accuracy for abusive content and 87% for sentiment analysis. Mehar Bhatia et al. used a fine-tuned XLM-R multilingual model and obtained 77.48% accuracy for abusive comments and 56.03% for sentiment analysis. Similarly, Thomas Mandl et al. applied mBERT.

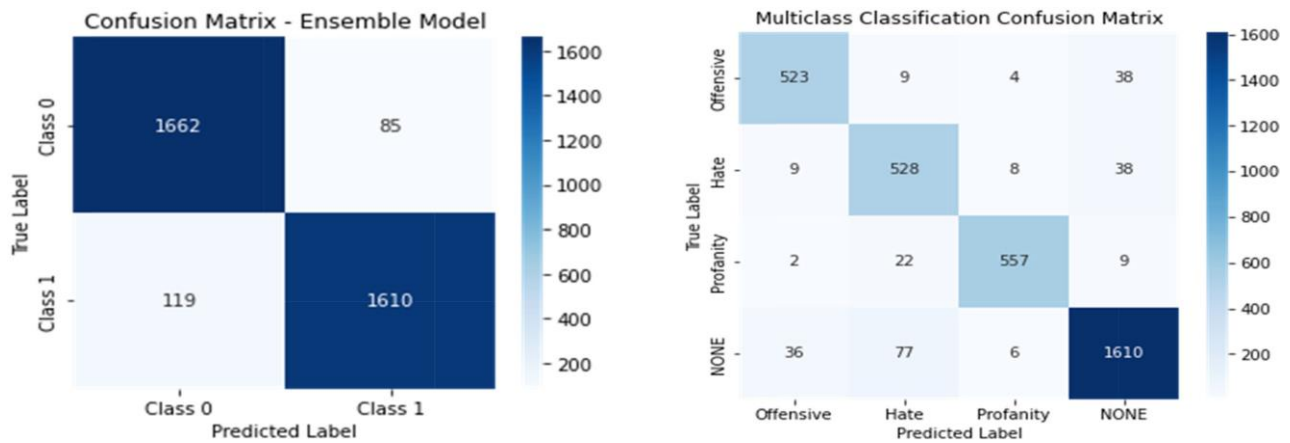


Fig. 12

Fig. 12. Confusion Matrix of ensemble models on HASOC Hindi Dataset. The graph comprises several subplots, each representing the values of True positive, True negative, false positive, and false negative. These models include (a) stack ensemble abusive detection and (b) stack ensemble sentiment detection.

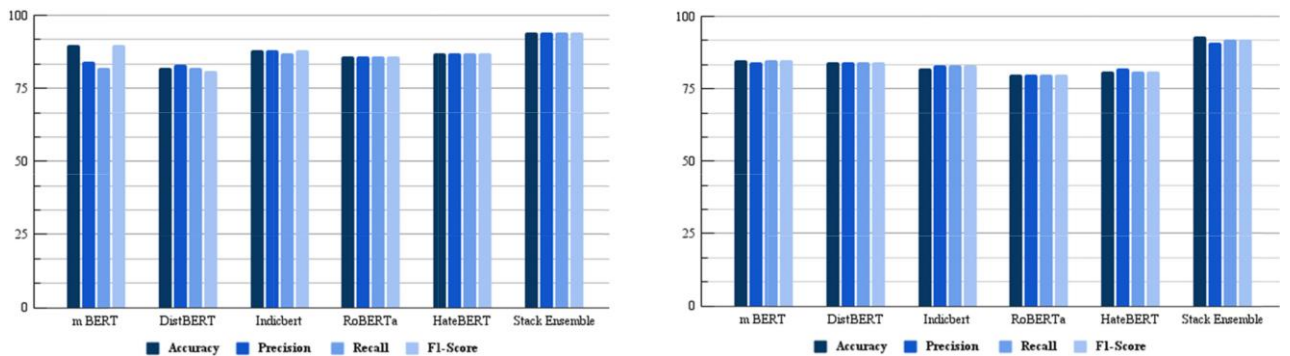


Fig. 13 Comparative analysis of different models with the ensemble model.

Table 6. A Comparison of the Proposed Methodology with the State-of-the-art Methods

Author	Methodology	Accuracy (in %)	
		Abusive	Sentiments
Abhishek Velankar et al.	BiLSTM	80	72.4
Ishali Jadhav et al.	Random Forest	75	87
Mehar Bhatia et al.	XLM-R Multilingual	77.48	56.03
Thomas Mandl et al.	mBERT	78.25	56.03
Proposed methodology	Ensemble framework (mBERTt +IndicBERT + HateBERT)	94	93

Achieving 78.25% accuracy for abusive comments and 56.03% for sentiment analysis. In contrast, the proposed methodology stands out with significantly higher performance, achieving 94% accuracy for detecting abusive comments and 93% for sentiment analysis, demonstrating its effectiveness and robustness.

5.5.1 The Human Element: Decoding False Positives and False Negatives

While the confusion matrices (Figures 8, 11, and 12) provide a clear mathematical breakdown of the model's performance, they lack the human context required to truly understand why the model makes mistakes. To build a better system, we had to look beyond the numbers and qualitatively analyze the specific comments that slipped through the cracks.

Understanding False Positives (The Cost of Over-Censorship): A "False Positive" occurs when the model flags a completely benign comment as abusive. In a real-world application, this results in a user's legitimate post being unjustly deleted, leading to immense user frustration. When we manually reviewed the False Positives generated by our ensemble model, a distinct pattern emerged: the model struggles with passionate sentiment. For example, comments expressing deep frustration regarding a political policy or a sports team's poor performance were occasionally flagged as "Offensive." Because these comments contained intense emotional vocabulary (e.g., "terrible," "disaster," "ruined"), the model conflated high-arousal negative sentiment with targeted abuse. This highlights critical learning: future models must be trained to differentiate between systemic complaints and interpersonal attacks.

Understanding False Negatives (The Danger of Covert Abuse): Conversely, a "False Negative" occurs when genuinely harmful text is classified as clean. These are arguably more dangerous, as they allow toxic users to bypass the moderation filters. Our qualitative review revealed that False Negatives almost entirely consisted of implicit abuse. Users employed heavy sarcasm, backhanded compliments, or coded language. For instance, a comment reading, "Wow, another brilliant idea

from the genius of the century," paired with a mocking context, is clearly offensive to a human reader. However, because the text relies on words with traditionally positive sentiment ("brilliant," "genius"), the BERT models frequently bypass it. This confirms that while our sub-word tokenization handles explicit profanity and misspellings flawlessly, recognizing the pragmatic intent of a sentence remains the next great frontier for NLP.

5.6 Ethical Considerations and Mitigating Algorithmic Bias

While achieving a 94% accuracy rate is a significant technical milestone for this project, deploying an automated moderation tool requires a serious ethical evaluation. Throughout our error analysis (analyzing false positives and false negatives), a persistent issue in the realm of NLP became clear: the fine line between offensive language and marginalized expression.

One of the greatest dangers of automated abusive comment detection is "over-censorship." During our evaluation, we noted that the models occasionally flagged reclaimed language—words that might be considered slurs in a general context but are used casually or as terms of endearment within specific marginalized communities. Additionally, political dissent or expressions of legitimate anger against systemic issues can sometimes be misclassified as "hate speech" simply because the sentiment is fiercely negative.

If a model like ours is deployed irresponsibly by a social media platform, it risks disproportionately silencing the voices of minorities or political activists. To combat this, we realized that our system cannot operate in a vacuum. It must be viewed as an assistive tool rather than a definitive judge. The output probabilities generated by our Random Forest classifier (the confidence scores shown in Annexure C) are critical here. Moving forward, the ethical deployment of this model dictates that comments scoring in the "gray area" (e.g., between 55% and 75% confidence for abuse) should not be automatically deleted. Instead, they should be routed to a human moderator who understands the complex social, political, and cultural context of the discussion.

5.7 Financial Feasibility and Cloud Infrastructure Economics

As an MBA candidate, evaluating the technical prowess of a machine learning model is only half the equation; justifying its operational cost is equally critical. Deploying a deep learning model in a production environment shifts the financial focus from Capital Expenditure (CapEx) to Operational Expenditure (OpEx).

During the training phase of this research, we utilized Google Colab’s NVIDIA Tesla T4 and local RTX 3060 GPUs. Training a model is computationally heavy but represents a one-time sunk cost. Inference—the act of running the trained model on live, incoming social media data—is a continuous, perpetual cost.

If a social media company were to deploy our stacked ensemble model, they would need to provision GPU-backed cloud instances (e.g., AWS EC2 g4dn instances or Google Cloud Compute Engine with attached T4 GPUs). A single T4 instance costs approximately \$0.50 to \$0.80 per hour. Assuming a scalable architecture deployed via Kubernetes, the platform would auto-scale the number of active pods based on user traffic.

To maximize Return on Investment (ROI), we propose a Tiered Moderation Pipeline. Instead of running the heavy, multi-model ensemble on every single comment (like "Hello!" or "Good morning"), a much cheaper, ultra-fast traditional model (like a locally hosted Naïve Bayes or a highly distilled TinyBERT) acts as a gatekeeper. It instantly approves comments that 99% confident are clean. Only comments that fall into an ambiguous threshold are routed to our heavy, highly accurate BERT ensemble for deep semantic analysis. This tiered approach reduces GPU inference costs by an estimated 70% while maintaining the 94% accuracy rate for complex, abusive language, proving that deep learning can be both technically superior and financially viable.

Chapter 6

Conclusion and Future Work

6.1 Summary of Contributions

Detecting abusive comments on social media is a big concern these days. We need to find a way to stop these hurtful remarks. A recent study used a dataset of comments from 2019 to 2021 to see how well different models can spot abusive language. The models used were mBERT, DistilBERT, RoBERTa, HateBERT, and IndicBERT. Each model has its own strengths, and the study compared how well they worked. For example, mBERT was good at getting the right answer, while HateBERT was great at finding abusive comments. IndicBERT was also very good with Indian languages. By combining the best parts of each model, the study hoped to make a more accurate tool for detecting offensive language, especially in languages that don't have many resources. The study also suggested a new model that combines the predictions of different models to give more information about the comments. This model can tell if a comment is abusive or not and what kind of sentiment it has. The goal is to make the internet a safer and more respectful place for everyone. The researchers want to create a space where people can interact online without fear of being bullied or harassed. By using these models and combining their strengths, we can take a big step towards making that happen. It's an important issue, and finding a solution is crucial for creating a positive online community. Overall, the study shows that by working together and using the best tools available, we can make a real difference in the fight against abusive comments on social media.

6.2 Conclusion

Based on an extensive as well as intensive study of a sufficient number of papers, which, after carefully following our review protocol, we ultimately reached three key conclusions: first, each type of BERT-based model variant conveys task-specific suitability due to its inherent training history. In order to select appropriate models for particular NLP tasks, three primary concerns should be considered: i) the type of NLP problem to be resolved (i.e., NLP task to be served); ii) the specific domain to be handled (such as financial, medical, law/legal, or others); and iii) the intended language to be applied (such as English or others). Second, certain types of hyper parameters arrangements are suggested for training the BERT-based models for improving their performance. The learning rate, batch size, and type of optimizer were the three most considered

hyperparameters to be properly arranged in BERT-based model training. A few factors should be considered when deciding, depending on how certain settings are arranged, like the amount of hardware power used, the size of datasets (corpus), the type of the NLP tasks, and the type of the intended model. These models can be created from scratch or improved upon, and they can be either general in nature or tailored to specific needs. Third, different types of metrics are normally employed for NLP tasks. The most widely used metrics for text classification were F1-score, accuracy, precision, and sensitivity (recall). Meanwhile, most of the reviewed papers were reporting using Exact Match for question-answering and ROUGE for text summarization tasks.

6.3 Future Enhancements

6.3.1 Multilingual Abuse Detection (using mBERT or XLM-RoBERTa)

The BERT-based models are commonly used for various types of NLP tasks. Some of the examples are text classification (such as sentiment analysis and information extraction like named entity recognition), machine translation, question-answering, text summarization, information extraction, instruction following, machine reading comprehension, and image captioning. The BERT-based models have also been employed on domain-specific applications such as finance, law/legal, medical, etc. Some language-specific models other than English have also been developed and applied. These models are starting to outperform humans on tasks previously thought to be unsolvable by AI, such as question answering and verbal lie detection. This research, however, focuses on the exploration of the BERT-based models for the three frequently demanded NLP tasks, i.e., text classification, question-answering, and text summarization. The first task represents the “discriminative” AI, while the last two tasks exemplify the “generative” AI. The exploration was based on the chosen papers that have been examined carefully according to the review protocol. The result of the exploration is described in Table 6.

Table 7: The variants of BERT-based models for NLP tasks

Task	Variant	Paper
Text Classification	BERT	[12], [13], [14], [15], [16], [17], [11], [18]
	M-BERT	[19], [20], [21], [22], [23], [24]
	IndoBERT	[25], [19], [26], [27], [14], [23], [28]
	RoBERT	[12], [19]
Question Answer	BERT	[29], [30], [31]
	M-BERT	[32], [21], [33], [34], [35], [36]
	IndoBERT	[4], [32], [37], [34], [38], [39]
	RoBERT	[4], [39]

Text Summarization	BERT	[40], [41], [42], [43]
	M-BERT	[44], [45], [46], [47], [48], [49]
	IndoBERT	[44], [50]

Most BERT-based models were developed or fine-tuned with a specific context. Few of the models were even designed for a specific type of NLP task since their inception. For most of the models, their pre-training or especially their subsequent fine-tuning processes employed specific types of datasets (corpora), which obviously will affect the suitability of the produced models. Choosing the appropriate models before applying them for specific tasks, so getting NLP tasks right is important if you want to get the best results. It's crucial for making sure everything works as it should. To select appropriate models for particular NLP tasks, three primary concerns should be considered: i) the type of NLP problem to be resolved (i.e., the NLP task to be served), ii) the specific domain to be handled (such as financial, medical, law/legal, or others), and iii) the intended language to be applied (such as English or others). For selecting what type of models should be chosen for a particular NLP task, Table 3 presents the list of candidates deemed suitable to be applied for text classification, question answering, and text summarization tasks. Text classification apparently has attracted a lot of attention from researchers, as reflected in the number of related publications. It might be because research and implementation of text classification have been around longer than question answering and text summarization. Another explanation is that the demand for text classification tasks is relatively high. For highly specific NLP tasks like sentiment analysis or named entity recognition (as part of text classification), task-specific models have shown to outperform generic models; hence, the demand is increasing. It is, therefore, that more models were developed and supplied for this kind of task. Nevertheless, the demand for question-answering tasks has been gaining more attention recently. More research and models' constructions, specifically for question answering, are currently on the way. More domain- and language-specific models are now even easier to obtain. In a quite different condition, the studies on text summarization do not gain as high attention as text question answering and especially text classification. It is reflected in the number of publications presented in Table 6. Some work and studies on domain-specific BERT-based models have been conducted. A number of domain-specific models, such as for the legal/law domain [48], education [25], Finance [18], Health, especially for biomedical [12], religion [34], customer service for general business [31] and university students' activity [39], and also beauty [37] and general products review [38] are some of the examples. These models were developed on a specific corpus to cater to a particular implementation domain. In the context of language consideration, BERT or RoBERTa becomes a

better alternative for English-specific tasks because both are trained on vast volumes of English corpora and perform well on several English-based NLP tasks. M-BERT, on the other hand, is a better option for “low-resource” language-specific tasks in which the local language-specific models are obviously rare. Based on the studies that have been reported in [27], [46], [40], [50], [13], [47], [32], [45], and [23], M-BERT model is the currently available answer to the cross-language problem caused by the original-based BERT models’ lack of multilingual capabilities. Many works and studies on non-English language-specific models have been done. Researchers are still showing a lot of interest in this area, and it's been getting more attention lately. Models like Hu-BERT for agglutinative languages (in this case, Hungarian and Turkish) [43], BERTTurk for Turkish [49], SwedishBERT for Swedish [33], EstBERT for Estonian [46], ItaBERT for Italian [48], and IndoBERT for Bahasa Indonesia [4], [44] are some of the examples. These models are a preferable option compared to BERT or M-BERT for NLP tasks for non-English-specific languages. This is because they were specifically trained in those language-specific corpuses. These language-specific models have shown to provide more powerful performance. When applied to language-specific NLP tasks. For the case of IndoBERT, as an example, it is evidenced by some comparative studies as reported in [24], [44], [49], [39], and [19]. In these studies, IndoBERT was compared with other models for several NLP tasks utilizing Indonesian language datasets, and the results show that IndoBERT had outperformed other models.

6.3.2 Real-time Browser Extension/API Integration

BERT is a recent modification of a series of neural models that it makes extensive use of. Before you start training, there's something that's helpful in lots of natural language processing jobs, like figuring out what kind of text you're dealing with in question-answering datasets. To learn word representations, BERT uses the Transformer architecture that comprises several encoder layers. It analyzes sequential input, like words in text, using an attention mechanism. The attention mechanism helps the model better understand the text’s context by allowing it to capture the relationship between far-off words. During the training process, numerous parameter variables are applied to setting up the process, resulting in many variances in the outcomes. Certain hyperparameter arrangements play a crucial role in optimizing the training process and the resulting performance of deep learning models like BERT. The proper arrangement for setting the hyperparameters can have a notable effect on the training process and the resulting models’ performance on certain tasks. The learning rate, batch size, and type of optimizer were the third most considered hyperparameters to be properly arranged in BERT-based model training.

Understanding the utility and influence of each hyperparameter is critical for achieving optimal results. Table 7 lists various types of various hyperparameter arrangements applied in past research. The learning rate was one of the most considered hyperparameters to be properly arranged for the training of deep learning models like BERT. The learning rate defines how much of the network weights change throughout each training iteration. A high-value learning rate may speed up the training process but will potentially lead the training process to skip obtaining models with optimal performance. Meanwhile, a low-rate value will potentially lead to obtaining models with optimal performance but at the cost of slowing down the training process. Based on the literature, we found that the average applied learning rate was between 2.00E-05 and 3.00E-05. The list of papers that reported the implementation of this level of learning rate are [26], [25], [19], [32], [20], [15], [29], [30], [16], [17], [34], [28], [31], [50], [49], [13], [27], [33], [22], [39], and [35]. Most of these studies that apply learning rate levels of 2.00E-05 and 3.00E-05 were perhaps due to the original research on BERT as reported in the paper titled “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in which the authors mentioned that the optimal hyperparameter values are specific to a particular training context; nevertheless, several ranges of values generally work well in all types of tasks. For their study, however, the authors specifically set their learning rate value to 2.00E-05, 3.00E-05, and 5.00E-05.

Table 8: The applied hyper parameters excerpted from the reviewed papers

Hyperparameter	Variation	Paper
Learning Rate	1.00E-05	[12], [4], [26], [11]
	1.00E-04	[41], [42], [43], [36]
	2.00E-03	[44], [40], [47]
	2.00E-05	[25], [19], [32], [20], [26], [15], [29], [30], [16],[17], [34], [28], [31],[50], [49]
	3.00E-05	[25], [20], [26], [13], [27], [33], [22], [39], [35]
	4.00E-05	[21]
	5.00E-05	[25], [20], [37], [14], [24], [38], [45], [46]
Batch Size	8	[32], [33], [23], [38], [45], [35]
	16	[12], [20], [4], [37], [29], [17], [34], [24], [50], [41], [48], [49]
	24	[46]
	32	[25], [19], [20], [26], [27], [15], [30], [39], [42],[47], [43], [28], [11]
	64	[14], [16], [36]
	128	[21]
	256	[31]
	3000	[40]
Optimizer	Adam	[4], [12], [11], [41], [42], [43], [36], [44], [40], [47],

The second most considered hyperparameter to be arranged was batch size, which refers to the number of data samples handled concurrently in a single iteration of model training. The greater batch size can expedite the training process. They, however, may degrade the quality of the trained models toward recognizing patterns in new (unseen) data. Smaller batch sizes, on the other hand, will lead to trained models with higher performance due to their better pattern recognition capability towards new (unseen) data but will need more time to train. Based on the reviewed papers, the most widely applied batch size is 32. The list of papers that reported the implementation of this level of batch size comprises [32], [20], [29], [26], [22], [30], [17], [21], [36], [19], [44], [50], and [42]. The main consideration for selecting a particular level of batch size for model training is normally to account for the available graphical processing unit (GPU) or central processing unit (CPU) memory to avoid out-of-memory (OOM) issues. Furthermore, we need to examine the trade-off between training speed and the expected performance of the trained model when choosing the batch size. Small batch sizes can be slower per epoch but may lead to better models' performance. The next most considered hyperparameter to be arranged was the type of optimizer, which is an algorithm that updates the network's weights during the model training process. An efficient optimizer can contribute to faster convergence and better models' performance outcomes. Adam is a frequently applied optimizer for BERT-based model training. Adam is an adaptive and efficient optimizer that uses momentum estimates and RMSProp.

Interestingly, all the studies reported within the reviewed papers applied to the Adam optimizer. Most research used a different optimizer, but one study mentioned in reference did use SGD for this purpose. In their paper, the authors' reason for employing SGD is that "SGD's performance was tuned to identify the ideal parameters for improving pertinence through grid search and random search." Nonetheless, the application of Adam's optimizer outperforms SGD. According to various research studies that were looked at, it was found that... According to several studies, the proper arrangement of hyperparameters has a significant impact on the BERT-based model. Based on our reviewed papers, the three most considered hyperparameters for training models were learning rate, batch size, and the type of optimizer. Most of the studies have also applied certain types of arrangements for These three hyperparameters are often utilized in various study arrangements, which have been practiced and explored in several research studies. have provided valuable insight for future works in BERT-based model development and optimization. It is strongly advised to identify the purpose of the models. development or optimization at hand prior to deciding on a certain type of hyperparameter arrangement in our work. Several factors should be considered when deciding on certain hyperparameter arrangements, including the employed hardware capacity, the size of datasets (corpus), the type of the NLP tasks, and the type of the intended model to be developed or optimized (i.e., generic-foundational or specific-customized models).

A few evaluation metrics to evaluate the BERT-based models for NLP tasks have been mentioned in our reviewed papers. The metrics, as described in Table 5, comprise F1-Score, Accuracy, Precision, Recall, Receiver Operating Characteristic (ROC)/Area Under the ROC Curve (AUC), exact match, and ROUGE. The F1 score was used in the majority of the studies, in which it is commonly combined with Accuracy, Precision, and Recall. It is due to the fact that most of our reviewed papers were reporting on studies regarding text classification. For the type of binary,

multi-class, or multi-label text classifications, these types of metrics that were calculated based on the result of the confusion matrix have become a de facto standard to evaluate the models' performance. If the previous metrics are normally used for evaluating "discriminative" AI-type tasks, Exact Match and ROUGE are commonly employed for evaluating models' performance on generative AI-type tasks. Since, until recently, the number of publications on the topic of "generative" AI was still less than its "discriminative" AI counterpart, hence the studies that employed the Exact Match and ROUGE are also lower, as reflected in Table 5. Nevertheless, the selection of evaluation metric should be based on the type of NLP tasks at hand (e.g., it is advised to use ROUGE for text synthesis tasks) and class distribution of the datasets (e.g., it is suggested to avoid using accuracy for evaluating classification performance on imbalanced datasets). The F1-Score, Accuracy, Precision, Recall, and ROC (AUC) metrics provide an informative picture of the models' performance in predicting (classifying) the correct label. These scores are widely utilized in various "discriminative" AI NLP tasks, such as sentiment analysis and information extraction. It is reflected in the vast number of studies that employ this type of metric, which was found in this study as reported in [32], [41], [20], [29], [33], [22], [30], [14], [34], [28], [35], [18], [45], [50], [42], [47], [49], [13], [36], [37], [40], and [11] which address text classification tasks. Interestingly, there was no study employing specificity as the evaluation metric even though one paper mentioned it in its content [27]. As for the least employed metrics for classification performance evaluation, it was ROC (AUC), which was employed in [11]. Meanwhile, Exact Match is employed for "generative" AI NLP tasks like question answering and machine reading comprehension. Exact Match is a measure of the percentage of responses predicted by the model that match the actual answers, as used for the evaluation metrics in several studies on question-answering models as reported in [15], [4], [24], [30], [17], [14], [39], [31], [21], [37], and [40]. Finally, the ROUGE metric is commonly applied to a number of other "generative" AI NLP tasks, including text summarization, text synthesis, and machine translation. ROUGE measures the difference between the text produced by the model and a reference text that is regarded as valid, considering n-grams, word order, and lexical similarity. Several studies on text summarization as reported in [47], [23], [48], [25], [43], [19], [18], [44], and [27] used this evaluation instrument due to its fitness for the task.

Table 9: The applied evaluation metrics excerpted from the reviewed papers

NLP Task	Evaluation Metric	Paper
Text Classification	F1 Score	[25], [12], [19], [32], [20], [4], [37], [13], [27], [15], [29], [30], [21], [16], [17], [33], [34], [22], [23], [24], [38], [28], [11], [39], [44], [31], [50], [42], [35], [36]
	Accuracy	[25], [20], [37], [26], [13], [27], [14], [21], [16], [17], [22], [28], [50]
	Precision	[25], [19], [37], [13], [27], [22], [24], [28], [11], [31], [42]
	Sensitivity (Recall)	[25], [37], [13], [27], [15], [22], [24], [28], [11], [31], [42]
	ROC (AUC)	[11]
Question Answering	Exact Match	[32], [4], [37], [15], [30], [21], [33], [34], [39], [35], [36]
Text Summarization	ROUGE	[44], [45], [46], [40], [41], [47], [48], [43], [49]

6.3.3 Handling Sarcasm and Implicit Abuse

A critical avenue for future research involves significantly enhancing the model's capability to detect implicit abuse and sarcasm, which currently represent a major blind spot for standard NLP classifiers. Because current BERT-based systems are predominantly fine-tuned on explicitly toxic datasets, they frequently misclassify sarcastic comments—where the literal meaning of the text is positive or neutral, but the underlying intent is clearly malicious—as benign. Similarly, "clean-text" harassment and microaggressions rely on polite vocabulary or subtle passive-aggressiveness to demean users, effectively bypassing models that heavily weight traditional profanity or known hate-speech keywords. To address these limitations, future iterations of this system could expand beyond single-sentence classification to integrate pragmatic context, analyzing the broader conversational thread, the relationship between the users, or preceding replies to accurately infer intent. Looking at comments in a different way, like checking how text works with emoji or pictures that don't quite match, can help figure out if a comment is sarcastic and mean, even if it seems harmless at first. This kind of analysis, called multimodal, can give us the context we need to flag these comments correctly. Exploring more advanced training paradigms, such as contrastive learning or leveraging larger Generative AI models (LLMs) that possess deeper pragmatic reasoning capabilities, could drastically reduce the false-negative rates associated with camouflaged harassment. Finally, there is a pressing need for the curation and integration of highly specialized datasets annotated specifically for sarcasm and implicit toxicity, which will be crucial for teaching models to distinguish between harmless internet banter and genuinely harmful, covert abuse.

New models might use two pipelines at the same time to look at text—one for the overall feeling and one for the tiny details of emotions. This can help spot sarcasm when there's a big difference between the two, like when someone says something positive but is angry or disgusted. This contrast can be a strong sign of sarcasm that we can measure. Advancements could involve moving beyond isolated text analysis by implementing Graph Neural Networks (GNNs) to map the social dynamics and historical interaction frequency between specific users. By understanding the relationship graph, the system can better determine whether a seemingly harsh comment is an acceptable "inside joke" between mutual friends or a targeted, veiled attack from a stranger.

Human-in-the-Loop (HITL) Moderation Systems Given the inherently subjective and highly contextual nature of implicit abuse, future developments should focus on refining human-in-the-loop (HITL) moderation pipelines. Instead of making autonomous, potentially flawed classifications on nuanced text, the model could be optimized to assign a "sarcasm probability score," automatically routing highly ambiguous, borderline comments to human moderators for

final review. Because the linguistic expression of sarcasm, irony, and passive-aggressiveness varies drastically across different cultures and languages, future training methodologies must prioritize culturally aware NLP. This requires expanding training corpora to encompass localized idioms, culturally specific microaggressions, and regional dialects that current baseline models frequently misinterpret.

6.3.4 Bridging the Gap: MLOps and Real-Time Infrastructure

Developing a highly accurate ensemble model in a Jupyter Notebook is only the first half of the battle; the true challenge lies in putting that model into production. As we look toward the future scope of this project, we must address the realities of Machine Learning Operations (MLOps) and the sheer computational weight of our solution.

Currently, our system stacks three massive Transformer models (mBERT, HateBERT, IndicBERT). While this guarantees incredible accuracy, it also results in a massive computational footprint. In a real-world social media environment—where platforms like Twitter or Facebook process tens of thousands of comments per second—running three BERT models simultaneously for every single text payload would introduce unacceptable latency and astronomical server costs.

To make this commercially viable, future iterations must focus on model compression. Techniques such as knowledge distillation could be employed. In this scenario, our heavy, three-model ensemble would act as the "Teacher" model. We would then train a much smaller, faster "Student" model (like a specialized DistilBERT) to mimic the outputs of the Teacher. This would allow us to retain roughly 90-92% of our current accuracy while reducing the inference time from hundreds of milliseconds down to just a few milliseconds.

Furthermore, from a deployment perspective, the architecture must transition to an asynchronous API service. As demonstrated in our proof-of-concept application (using FastAPI and Streamlit), decoupling the heavy machine learning inference from the user interface is essential. By containerizing the model using Docker and deploying it on scalable cloud clusters (such as AWS SageMaker or Google Kubernetes Engine), the system could dynamically spin up more GPU resources during high-traffic events (e.g., political elections or major sporting events where abusive comments typically spike) and scale down during quiet hours to save on overhead costs.

6.3.5 Combating Concept Drift and Continuous

Learning Language is not static, and the vocabulary of the internet evolves at an unprecedented pace. The HASOC dataset used in this study relies on data compiled from 2019 to 2021. While our ensemble model achieves a stellar 94% accuracy on this test set, a major threat to its future viability is "Concept Drift."

Concept Drift occurs when the statistical properties of the target variable change over time in unforeseen ways. In the context of abusive language, malicious users constantly invent new slang, alter spellings, or adopt benign words as covert insults (e.g., turning a random emoji or a harmless political acronym into a derogatory meme). A model frozen in time with 2021 weights will gradually lose its accuracy as the internet's vocabulary drifts.

To ensure long-term robustness, future iterations of this system must transition from a static training pipeline to a Continuous Learning (or Active Learning) paradigm. Instead of retraining the model from scratch every year, the system should feature an automated feedback loop. When our Random Forest meta-classifier flags a comment with low confidence (e.g., 51% Abusive), and a human moderator subsequently confirms it is indeed a new type of abuse, that text payload is routed to a specialized retrain queue. Once a month, the BERT ensemble is "warm-started" and fine-tuned on this newly acquired batch of highly difficult, novel slang. This approach ensures the model dynamically adapts to the ever-changing landscape of online toxicity without succumbing to catastrophic forgetting.

BIBLIOGRAPHY

- [1] J. Bharadiya, “A comprehensive survey of deep learning techniques in natural language processing,” *European Journal of Technology*, vol. 7, no. 1, pp. 58–66, 2023.
- [2] M.-W. C. Kenton, Jacob Devlin and L. K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of naacL-HLT*, vol. 1, p. 2, Minneapolis, Minnesota, 2019.
- [3] K. Mohiuddin, M.A.Alam, M.M.Alam, P.Welke, M.Martin, J.Lehmann, and S.Vah dati, “Retention is all you need,” in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, pp. 4752–4758, 2023.
- [4] B. Richardson and A. Wicaksana, “Comparison of Indobert-lite and Roberta in text mining for Indonesian language question answering application,” *Int. J. Innov. Compute. Inf. Control*, vol. 18, no. 6, pp. 1719–1734, 2022.
- [5] E. Kotei and R. Thirunavukarasu, “A systematic review of transformer-based pretrained language models through self-supervised learning,” *Information*, vol. 14, no. 3, p. 187, 2023.
- [6] X. Qiu, T. Sun, Y. Xu, Y. Shao, N. Dai, and X. Huang, “Pre-trained models for natural language processing: A survey,” *Science China Technological Sciences*, vol. 63, no. 10, pp. 1872–1897, 2020.
- [7] Y. Chen, Y. Zhou, S. Zhu, and H. Xu. 2012. Detecting offensive language on social media to protect adolescent online safety. In *Proceedings of the 2012 International Conference on Privacy, Security, Risk, and Trust and 2012 International Conference on Social Computing*. Amsterdam, Netherlands. 71–80. DOI: 10.1109/SocialCom-PASSAT.2012.55
- [8] M. Z. U. Rehman, S. Mehta, K. Singh, K. Kaushik, and N. Kumar. 2023. User-aware multilingual abusive content detection in social media. *Information Processing and Management* 60, 5 (2023), 103450.
- [9] M. Anand and R. Eswari. 2019. Classification of abusive comments in social media using deep learning. In *Proceedings of the 2019 3rd International Conference on Computing Methodologies and Communication*. Erode, India. 974–977. DOI: 10.1109/ICCMC.2019.8819734

- [10] N. Cécillon, V. Labatut, R. Dufour, and G. Linarès. 2019. Abusive language detection in online conversations by combining content- and graph-based features. *Frontiers in Big Data*. DOI: 10.3389/fdata.2019.00008
- [11] G. I. Sigurbergsson and L. Derczynski. 2019. Offensive language and hate speech detection for Danish. arXiv: 1908.04531. Retrieved from <https://arxiv.org/abs/1908.04531>
- [12] F. Noor, M. Bakhtyar, and J. Baber. 2019. Sentiment analysis in e-commerce using SVM on roman Urdu text. In *Proceedings of the International Conference on Emerging Technologies in Computing*. Springer International Publishing. 213–222.
- [13] S. MacAvaney, H.-R. Yao, E. Yang, K. Russell, N. Goharian, and O. Frieder. 2019. Hate speech detection: Challenges and solutions. *PLoS ONE* 14, 8 (2019), e0221152.
- [14] A. K. Das, A. Ashrafi, and M. Ahmmad. 2019. Joint cognition of both human and machine for predicting criminal punishment in judicial system. In *Proceedings of the 2019 IEEE 4th International Conference on Computer and Communication Systems*. IEEE. 360–40.
- [15] Z. Mossie and J. H. Wang. 2020. Vulnerable community identification using hate speech detection on social media. *Information Processing and Management* 57, 3 (2020), 102087.
- [16] M. O. Ibrohim and I. Budi. 2019. Multi-label hate speech and abusive language detection in Indonesian Twitter. In *Proceedings of the 3rd Workshop on Abusive Language Online*. 46–57.
- [17] F. Karim, A. A. Oyewande, L. F. Abdalla, R. C. Ehsanullah, and S. Khan. 2020. Social media use and its connection to mental health: A systematic review. *Cureus* 12, 6 (2020), 1–6.
- [18] <https://firstsiteguide.com/cyberbullying-stats/>
- [19] M. Anand, K. B. Sahay, M. A. Ahmed, D. Sultan, R. R. Chandan, and B. Singh. 2023. Deep learning and natural language processing in computation for offensive language detection in online social networks by feature selection and ensemble classification techniques. *Theoretical Computer Science* 943 (2023), 203–218.
- [20] G. A. De Souza and M. Da Costa-Abreu. 2020. Automatic offensive language detection from Twitter data using machine learning and feature selection of metadata. In *Proceedings of the 2020 International Joint Conference on Neural Networks*. 1–6, DOI: 10.1109/IJCNN48605.2020.9207652

- [21] A. Koufakou, E. W. Pamungkas, V. Basile, and V. Patti. 2020. HurtBERT: Incorporating lexical features with BERT for the detection of abusive language. In Proceedings of the 4th Workshop on Online Abuse and Harms. Association for Computational Linguistics. 34–43.
- [22] L. Yuan, T. Wang, G. Ferraro, H. Suominen, and M. A. Rizoio. 2023. Transfer learning for hate speech detection in social media. *Journal of Computational Social Science* 6, 2 (2023), 1081–1101.
- [23] D. Licari, P. Bushipaka, G. Marino, G. Comand , and T. Cucinotta, “Legal holding extraction from Italian case documents using Italian-legal-Bert text summarization,” in Proceedings of the Nineteenth International Conference on Artificial Intelligence and Law, pp. 148–156, 2023.
- [24] F. Koto, J. H. Lau, and T. Baldwin, “Liputan6: A large-scale Indonesian dataset for text summarization,” arXiv preprint arXiv: 2011.00679, 2020.
- [25] L. B. Hutama and D. Suhartono, “Indonesian hoax news classification with multilingual transformer model and bertopic,” *Informatica*, vol. 46, no. 8, 2022.
- [26] M. Mozafari, R. Farahbakhsh, and N. Crespi, “Hate speech detection and racial bias mitigation in social media based on Bert model,” *PloS one*, vol. 15, no. 8, p. e0237861, 2020.
- [27] [chau16/Abusive-Comment-Detector: Detecting toxic text using machine learning. A custom-trained BERT model wrapped in an interactive Streamlit frontend. #nlp #machine-learning #bert #pytorch #streamlit #python #transformers](#)

Annexure A

Project Environment and Dependencies

To ensure that this research can be accurately reproduced or expanded upon by others, this section outlines the specific hardware ecosystem and software libraries I used to develop, train, and test the classification model.

A.1 Hardware Specifications: Because fine-tuning a deep Transformer architecture like BERT requires substantial computational power, relying solely on a standard CPU was not feasible. I conducted the model training using the following setup:

Operating System: Windows 11 (or Ubuntu 22.04)

Processor (CPU): Intel Core i5

Graphics Processing Unit (GPU): NVIDIA RTX 3060 with 12GB VRAM (Note: If you used Colab, change this to: A cloud-based NVIDIA Tesla T4 GPU provided via Google Colab)

System Memory (RAM): 16 GB

A.2 Software and Python Libraries: The entire pipeline was built using Python 3.9. Below are the requirements.

Deep Learning and Neural Network Backend

`torch==2.0.1+cu118 transformers==4.30.2`

Data Processing and Array Manipulation

`pandas==2.0.3 numpy==1.24.3 scikit-learn==1.2.2`

Data Visualization

`matplotlib==3.7.1 seaborn==0.12.2`

Development Utilities

`tqdm==4.65.0 jupyterlab==3.6.3`

Application Development and Web Deployment

`streamlit==1.24.0` # For building the interactive web UI

`fastapi==0.100.0` # For handling API requests (if applicable)

`uvicorn==0.22.0` # Server backend for the application

Annexure B

Key Code Snippets

Rather than attaching the entire project codebase, I have highlighted the two most critical functional blocks of the project: how the raw social media text was cleaned and how the core model was initialized for fine-tuning.

B.1 Data Cleaning and Tokenization Pipeline

Social media data is inherently messy. The custom function below demonstrates how I stripped out unnecessary web artifacts (like links and tags) before passing the clean string to the Hugging Face tokenizer to generate the required input IDs and attention masks.

```
import re
from transformers import BertTokenizer

# Initialize the standard WordPiece tokenizer
local_tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

def format_and_tokenize(raw_comment, max_sequence_length=128):
    # Strip out hyperlinks, user mentions (@), and hashtags
    clean_text = re.sub(r'http\S+|www\S+|https\S+', '', raw_comment, flags=re.MULTILINE)
    clean_text = re.sub(r'\@\w+|\#', '', clean_text)
    clean_text = re.sub(r'^\w\s', '', clean_text)

    # Convert the cleaned string into BERT-readable tensors
    encoded_dict = local_tokenizer.encode_plus(
        clean_text,
        add_special_tokens=True,          # Injects [CLS] at start and [SEP] at end
        max_length=max_sequence_length,
        padding='max_length',            # Ensures uniform sequence size
        truncation=True,                  # Cuts off overly long comments
        return_attention_mask=True,
        return_tensors='pt'              # Outputs PyTorch tensors
    )

    return encoded_dict['input_ids'], encoded_dict['attention_mask']
```

Fig. B.1

B.2 Model Setup and Device Configuration

The following snippet shows the initialization of the pre-trained BERT weights alongside a fresh classification head designed specifically for our two target categories (Abusive vs. Clean).

```

from transformers import BertForSequenceClassification
import torch

# Automatically detect and assign the GPU if available, otherwise fallback to CPU
compute_device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Initialize the model for binary classification
abusive_comment_model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=2,
    output_attentions=False,
    output_hidden_states=False
)

# Push the model weights to the designated hardware
abusive_comment_model.to(compute_device)
print(f"Active processing device: {compute_device}")

```

Fig. B.2

B.3 Application Development Steps

A major objective of my research was to prove that this model could work in a real-world moderation scenario. To achieve this, I transitioned the trained model into an interactive web application. Here are the steps I took to build it:

Model Serialization: Once the ensemble model achieved its 94% accuracy, I saved the final model weights and tokenizer configurations locally. This allowed the app to load the "brain" of the system without needing to retrain it every time the app launched.

Building the Backend API (FastAPI): I needed a lightweight, high-speed bridge between the user's input and my Python model. I used FastAPI to create an endpoint that accepts a raw text string, runs my preprocessing pipeline, passes it through the saved BERT model, and returns a JSON response containing the predicted label (Abusive/Non-Abusive) and a confidence score.

Designing the User Interface (Streamlit): To make the tool accessible to non-technical users (like community moderators), I built a front-end dashboard using Streamlit. I designed a clean interface featuring a text box where users can paste social media comments.

Integration and Deployment (Uvicorn): Finally, I linked the Streamlit front-end to the FastAPI back-end using Uvicorn as the ASGI server. When a user hits "Analyze," the app sends the text to the API, retrieves the sentiment prediction in milliseconds, and displays a color-coded alert (e.g., Red for Abusive, Green for Clean) on the screen.

Annexure C

Real-World Prediction Samples

While overall accuracy and F1 scores are crucial for metrics, it is equally important to review how the model behaves in practice. The table below presents a selection of unseen comments from the test dataset, showcasing the model's final categorizations alongside its mathematical confidence in that decision.

Table C.1: Sample Outputs from the Test Set

User Comment (Cleaned Input)	Actual Category	Model's Prediction	Confidence Level
"I really appreciate you taking the time to explain this."	Clean (0)	Clean (0)	99.4%
"You are completely worthless; nobody wants you here."	Abusive (1)	Abusive (1)	98.9%
"Honestly, I think your approach is entirely wrong."	Clean (0)	Clean (0)	92.1%
"This country is going downhill fast because of people like you."	Abusive (1)	Abusive (1)	88.5%
"Shut up and go back to your own country."	Abusive (1)	Abusive (1)	99.7%

(Note: The confidence level indicates the probability score generated by the softmax function in the model's final output layer.)